

ReviewBuster: An NLP-Based Machine Learning System for Detecting Deceptive Hotel Reviews

31005 Machine Learning - Assignment 2

Overview

ReviewBuster examines whether hotel-review text can distinguish reviews labelled truthful from those labelled deceptive. A model flag could help readers or moderators investigate suspicious reviews, but a prediction cannot establish that an individual reviewer lied.

We removed four duplicate rows from a corpus of 1,600 reviews and reserved four entire hotels for final testing. Using the remaining 1,277 training reviews, we compared logistic regression, a linear support vector machine, and a one-hidden-layer neural network across five hotel-grouped validation folds and several TF-IDF text representations. Training-only tuning selected a linear SVM with non-lemmatised unigrams and bigrams at $C=6.0$.

On 319 reviews from the held-out hotels, the final model achieved 0.881 accuracy and 0.876 F1 for the deceptive class. It detected 134 of 160 deceptive reviews, missed 26, and incorrectly flagged 12 truthful reviews. These results describe performance on this corpus and four test hotels. Source-related writing patterns may influence the model because review source and label are confounded in the dataset.

Implementation Link

The implementation is organised into three Jupyter notebooks covering data exploration and preprocessing, model comparison, and final tuning and testing.

Link to Jupyter Notebooks:

https://github.com/nvmngo/ReviewBuster_NLP_MLproject

Contents

1 Introduction	3
1.1 Motivation and problem context	3
1.2 Project scope and research question	4
1.3 Learning task and system input/output	4
2 Data Exploration and Preparation	5
2.1 Dataset background and labels	5
2.2 Exploratory data analysis and data quality	6
2.3 Cleaning, hotel split, and leakage prevention	11
2.4 Text preprocessing and TF-IDF feature construction	14

Contents (continued)

3 Model Design and Cross-Validation	18
3.1 Shared pipeline, validation design, and metrics	18
3.2 Logistic regression: method and results	21
3.3 Linear SVM: method and results	26
3.4 MLP: method and results	31
3.5 Comparison across models and text representations	37
3.6 Primary pipeline selection	45
4 Hyperparameter Refinement and Final Test Evaluation	46
4.1 SVM hyperparameter tuning	46
4.2 Final training and held-out test procedure	49
4.3 Final test results	51
4.4 Error analysis and interpretation	56
4.5 Discussion: practical objective, limitations, and future work	60
4.6 Conclusion	63
5 Implementation Log	64
5.1 Development decisions, challenges, and solutions	64
5.2 Use of AI tools and verification	64
5.3 Knowledge gaps and attempts to resolve them	65
References	66

1 Introduction

1.1 Motivation and problem context

Most of my family, especially my brother and father, have long worked in the hotel industry. Hospitality businesses like this tend to grow really fast in Vietnam, and I can see why they are so dedicated to their jobs. Though it has been a while since I last lived with them in Vietnam, I still remember some of their frustrating conversations about trying to solve their hotels' review problems. Most of them were about trying to figure out whether a customer review about the hotel was genuine or not, since reviews are basically a representation of the business.

And now, I have entered university, studying for a degree in Computing Science and AI, basically following a completely different path from my brother and father. However, part of me still feels like contributing to the hospitality field in some way. This is where the idea for this project came from.

NLP is beautiful. There is just something beautiful about taking raw text and turning it into ugly numerical values that make almost no sense to me just by looking at them, yet a machine can work with them. The words are still familiar to me, but suddenly they are represented in a form that a model can use to learn patterns.

That made me wonder: could those patterns help with the same review problems my brother and father had been dealing with? Could something that starts as a few sentences on a hotel's page become useful information for identifying potentially deceptive reviews? I wanted to try it for myself, and that curiosity became ReviewBuster. It also felt like a small way of bringing my own path back to theirs.

I know a model cannot tell us for certain whether someone is lying, but perhaps it could help point out reviews that deserve a closer look. For me, that felt like a good place to start.

1.2 Project scope and research question

Those conversations stayed with me, and eventually became the starting point for ReviewBuster. I wanted to explore whether the words in a review could offer some help with the problem my family had been dealing with. Could a model learn useful differences between truthful and deceptive reviews, and recognise those differences when reading something new?

For this first version, I kept the project focused on hotel reviews. I used a dataset containing 1,600 reviews from 20 hotels in Chicago, each labelled truthful or deceptive. Although these hotels are far from the businesses my family works in, the dataset gave me a concrete place to begin investigating the question.

The main question I wanted to answer was:

How well can a machine learning model distinguish truthful from deceptive reviews when it encounters reviews of hotels it has not seen during training?

I also wanted to understand which choices would help it do that. I compared three models - logistic regression, a linear SVM, and a small neural network - and tried different ways of representing the text. These included looking at individual words, adding pairs of neighbouring words, and reducing words to their base forms. I kept four hotels aside for the final test, so the chosen model would have to work with reviews from unfamiliar hotels.

There is still a limit to what this experiment can tell us. The model learns from the labels and writing patterns in this particular dataset, whose collection sources are closely tied to those labels. A successful prediction could therefore be useful as a reason to look more closely at a review. It would not, by itself, settle whether the person who wrote it was being honest.

1.3 Learning task and system input/output

The overall idea behind ReviewBuster is fairly straightforward: give it a hotel review, turn the text into numerical features, and let a trained model predict whether the review is truthful or deceptive.

At a general level, the system follows this flow:

Review text $\rightarrow$ **Text preprocessing** $\rightarrow$ **Numerical representation** $\rightarrow$ **Trained classifier** $\rightarrow$ **Predicted label**

The input is the written review itself. The preprocessing stage prepares that text, and the feature extraction stage converts it into a numerical representation that the classifier can work with. During training, reviews with known labels provide examples from which the model learns patterns. Once trained, it uses those patterns to make predictions for reviews it has not seen before.

The output is one of two predicted labels: truthful (0) or deceptive (1). In practical terms, a deceptive prediction would flag the review for a closer look. It would still be a model's assessment based on the text, rather than a definite conclusion about the person who wrote it.

2 Data Exploration and Preparation

2.1 Dataset background and labels

To begin working on ReviewBuster, I needed a collection of hotel reviews with labels already available. I used a dataset downloaded from Kaggle, stored in the project as `deceptive-opinion.csv`. It contains **1,600 written reviews covering 20 hotels in Chicago**, with each review labelled either truthful or deceptive.

The dataset comes from the Deceptive Opinion Spam research corpus. The researchers collected reviews from online review platforms and recruited participants through Amazon Mechanical Turk to deliberately write deceptive reviews. These examples provided a way to study whether the language in a review could help distinguish the two classes. The collection process is described in the original studies by Ott et al. (2011) and Ott et al. (2013).

Each row in the CSV represents one review and contains five attributes. Some describe the review's background, while text contains the actual writing that the model will learn from.

Table 1. Dataset attributes, datatypes, and meanings

Attribute	Pandas datatype	Analytical type	Description
deceptive	object	Binary nominal category	The review's assigned label: truthful or deceptive. This is the target for classification.
hotel	object	Nominal category	The hotel associated with the review, such as hilton, allegro, or monaco.
polarity	object	Binary sentiment category, treated as ordinal in Notebook 1	Whether the review expresses a negative or positive opinion about the hotel.
source	object	Nominal category	The platform or collection source, recorded as MTurk, TripAdvisor, or Web.
text	object	Unstructured text	The complete written hotel review, including the writer's descriptions, opinions, and experiences.

As shown in Notebook 1, pandas reads all five columns as object because their values are stored as text. This storage datatype does not describe their analytical meaning: a hotel name identifies a category, while a review contains unrestricted language.

The two labels, deceptive and polarity, also describe different things. **Deception concerns the review's assigned authenticity label; polarity concerns the opinion it expresses.** A complaint can describe a genuine experience, just as praise can be fabricated. For ReviewBuster, the task is to predict the truthful or deceptive class from the written review.

2.2 Exploratory data analysis and data quality

Before training a model, I wanted to understand what was actually inside the dataset. Having reviews labelled truthful or deceptive gives us a starting point, but it does not automatically mean that the data is ready for a fair experiment. In Notebook 1, I examined missing values, repeated reviews, label distributions, review lengths, and vocabulary differences.

The statistics in this section describe the raw 1,600 reviews, including the four extra duplicate rows identified during exploration.

Data quality

The first check was whether any information was missing. Across all five columns, there were no missing values and no empty or whitespace-only values. This meant that every record contained review text, a target label, and the associated metadata.

However, the duplicate checks found four exact duplicate pairs. Eight rows were involved, but only four were additional copies. Checking the review text separately identified the same four pairs, and their other column values also matched. All four extra copies were truthful, negative reviews collected from Web.

These duplicates needed attention because repeated examples can receive extra weight during training and distort the reported sample counts. With a random split of individual rows, a review and its copy could also appear on opposite sides of the training/test split. Keeping reviews grouped by hotel prevents that particular overlap, but removing the extra copies still avoids counting the same review twice. The removal procedure is described in Section 2.3.

Label balance, hotel coverage, and sentiment

The raw dataset contains 800 truthful and 800 deceptive reviews. Each of the 20 hotels contributes 80 reviews, divided equally into 40 truthful and 40 deceptive examples. Therefore, neither target class dominates the raw dataset, and the hotel name alone does not identify the label.

Sentiment is also balanced. There are 800 positive and 800 negative reviews overall, and each target class contains 400 of each. This is useful because it challenges a simple assumption: a deceptive review is not necessarily positive, and a negative review is not necessarily truthful. In this dataset, sentiment polarity alone provides no separation between the two target classes.

These distributions support evaluating the classifier on unfamiliar hotels while keeping the prediction task focused on the review text.

The relationship between source and label

The source distribution revealed a more serious issue, as shown in Table 2.

Table 2. Review source and target label in the raw corpus (N = 1,600)

Review source	Truthful	Deceptive	Total
MTurk	0	800	800
TripAdvisor	400	0	400
Web	400	0	400

Every MTurk review is labelled deceptive, while every TripAdvisor and Web review is labelled truthful. In other words, source perfectly identifies the target label in this dataset.

2.2 Exploratory data analysis and data quality (continued)

Including source as a predictive feature would give the classifier a shortcut. It could produce apparently strong results by identifying where a review came from, without learning useful differences in the review text. Source is therefore retained for auditing and interpretation, and excluded from the model’s predictive inputs.

There is still a limitation after removing that column. Reviews collected from different sources may have different writing styles, and those differences remain inside the text. Consequently, a pattern associated with the deceptive label could reflect the collection process as well as deception. This affects how confidently we can interpret both the EDA findings and the later model results.

Review length

I measured review length using character counts, whitespace-separated word counts, and approximate sentence counts. Sentence counts were estimated from punctuation boundaries, so they should be treated as an approximation. Table 3 summarises the word-count results.

Table 3. Word-count statistics for the raw corpus (N = 1,600)

Review group	Mean words	Median words	Standard deviation	Minimum	Maximum
All reviews	148.78	128	87.34	25	784
Truthful	150.91	130	89.81	30	749
Deceptive	146.64	125	84.79	25	784

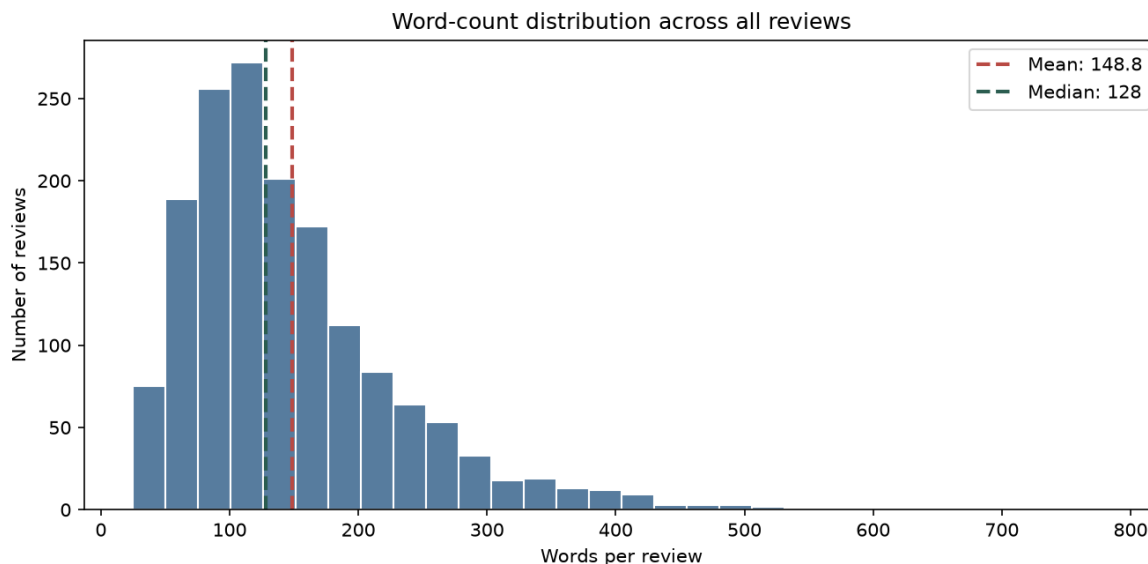


Figure 1. Word-count distribution from Notebook 1 (raw N = 1,600). The dashed lines mark the mean and median.

Figure 1 shows a distribution with a long right tail: most reviews are relatively short, while a smaller number of lengthy reviews pull the mean above the median.

2.2 Exploratory data analysis and data quality (continued)

Truthful reviews are slightly longer on average, but the difference is small compared with the variation within each class. Their middle 50% spans 89-183 words, compared with 88-182 words for deceptive reviews. The box plots in Figure 2 therefore show substantial overlap.

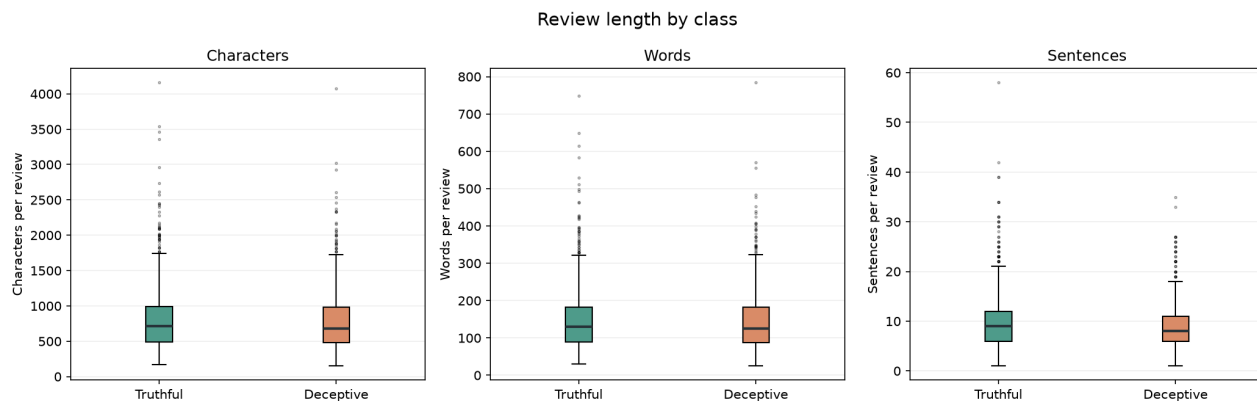


Figure 2. Review length by class from Notebook 1 (800 reviews per class). Characters, words, and approximate sentence counts overlap substantially; points beyond the whiskers show longer reviews.

Character and sentence counts tell a similar story. Truthful reviews average 821.02 characters and 10.10 sentences, compared with 791.77 characters and 9.02 sentences for deceptive reviews. These differences describe tendencies within this corpus; they do not establish a reliable rule for identifying deception. Review length alone does not appear to offer clear class separation.

Vocabulary and wording

The vocabulary exploration found 9,571 distinct words across the raw corpus. Truthful reviews contained 7,182 distinct words, deceptive reviews contained 6,325, and 3,936 appeared in both groups.

For this descriptive analysis, text was lowercased, punctuation was ignored, and one-letter tokens were excluded. Different word forms, such as room and rooms, were counted separately. Common English stop words were hidden from the displayed lists to make the hotel-related vocabulary easier to inspect; this was a display choice.

Both classes frequently mention hotels, rooms, stays, staff, and service. These shared words show why finding a particular hotel-related word is unlikely to be enough to classify a review. Some selected words nevertheless appeared at different rates, as shown in Table 4 and Figure 3.

2.2 Exploratory data analysis and data quality (continued)

Table 4. Percentage of raw reviews containing each selected word

Word	Truthful reviews (%)	Deceptive reviews (%)
chicago	44.0	77.0
experience	11.5	23.9
luxury	1.1	11.9
recommend	13.4	21.8
location	38.4	15.0
floor	19.5	6.2
great	41.4	27.1
bathroom	22.6	13.1

Each percentage counts reviews containing the word, rather than the number of times it was repeated. Both classes contain 800 reviews in this raw-corpus comparison.

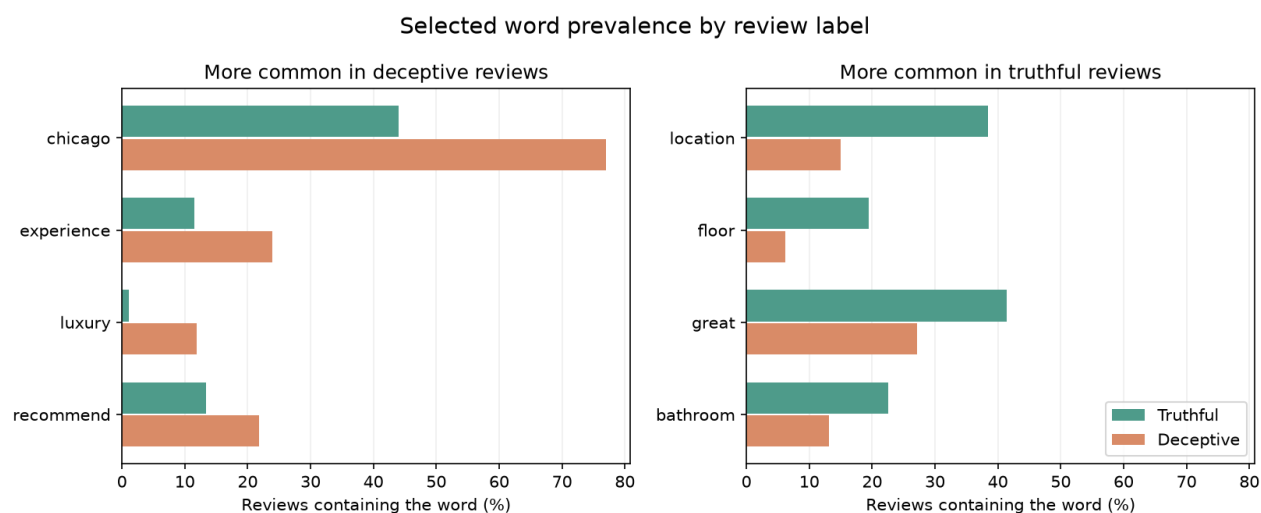


Figure 3. Selected word prevalence by review label, reproduced from Notebook 1. Each bar is the percentage of the 800 reviews in that class containing the word at least once.

Some deceptive reviews appear to use broader descriptions, such as experience and luxury, while truthful reviews more often mention details such as the location, floor, or bathroom. This offers a reason to investigate combinations of textual features through machine learning. However, these are descriptive associations. A truthful reviewer can write generally, and a deceptive reviewer can include specific details. The source-label relationship also prevents us from treating these words as universal indicators of deception.

2.2 Exploratory data analysis and data quality (continued)

Implications for the project

The exploration identified both useful structure and important complications. The class balance supports comparing performance across the two labels, but the overlapping lengths and vocabulary suggest that a simple rule based on sentiment, review length, or an individual word would be insufficient.

These findings motivated using multiple text features together, removing duplicate records, excluding source from prediction, and evaluating performance across separate hotel groups. They also established a limitation that must remain visible throughout the report: successful classification within this corpus does not necessarily demonstrate reliable deception detection in other review settings.

The vocabulary statistics above describe the full raw corpus. The predictive vocabulary and TF-IDF weights are learned separately from training reviews, including within each cross-validation training fold, as explained in the later sections.

2.3 Cleaning, hotel split, and leakage prevention

After exploring the dataset, the next step was to prepare it for modelling. I wanted the evaluation to answer the question introduced earlier: could the model recognise truthful and deceptive reviews about hotels it had not encountered during training? That meant deciding which information the model could use and keeping the training and test data separate.

Removing repeated records

The duplicate checks in Section 2.2 identified four additional copies of existing reviews. I removed these using `drop_duplicates()`, keeping one instance of each exact row. This left 1,596 reviews: 796 truthful and 800 deceptive. The original dataframe remained available for exploration, while the preparation steps worked on a separate copy.

Removing these copies prevents particular reviews from receiving extra weight during training or being counted more than once during evaluation. A random split of individual rows could also place a review and its copy in different sets. Our hotel grouping would already prevent that particular overlap, because both copies belong to the same hotel, but duplicate removal still gives more accurate training weights and evaluation counts.

The small difference between the two class totals did not justify adding or removing other reviews simply to restore equal numbers.

Encoding the target and separating metadata

I encoded the target as `truthful = 0` and `deceptive = 1`, stored as `int8`. The code checks for missing or unexpected labels before applying the mapping, so an unrecognised value cannot silently become a missing target. With this encoding, later metrics for the positive class refer to deceptive reviews.

The remaining attributes have separate roles:

- hotel stays as a string and determines which reviews belong together during evaluation.
- source stays available for auditing, but is excluded from predictive inputs because it directly identifies the target in this dataset.
- polarity is encoded as `negative = 0` and `positive = 1` for descriptive summaries. It is also excluded from predictive inputs, following the balanced sentiment distributions observed in Section 2.2.

The classifier therefore receives numerical features extracted from the review text. The target and metadata can remain beside those features in the stored tables for inspection, but they are separated when preparing model inputs.

Holding out complete hotels

I split the dataset by hotel rather than by individual review. Reviews from 16 hotels were retained for training and model selection, while reviews from four hotels were reserved for final testing.

The code sorts the hotel names and uses NumPy's random generator with seed 42 to select four hotels without replacement. This makes the selection reproducible. The selected hotels were Allegro, Homewood, Knickerbocker, and Monaco; their selection was not based on review wording, labels, or expected model performance.

Allegro, Homewood, and Knickerbocker each contribute 80 test reviews. Monaco contributes 79 because one additional duplicate was removed earlier.

2.3 Cleaning, hotel split, and leakage prevention (continued)

Table 5. Hotel split after duplicate removal (N = 1,596)

Dataset portion	Hotels	Reviews	Truthful	Deceptive
Training and model selection	16	1,277	637	640
Final test	4	319	159	160
Total	20	1,596	796	800

The split is exactly 80/20 by hotel count and approximately 80/20 by review count. All reviews from a hotel remain on the same side of the split. This matches the research question more directly than a row-level split, which could place reviews of the same hotel in both training and testing.

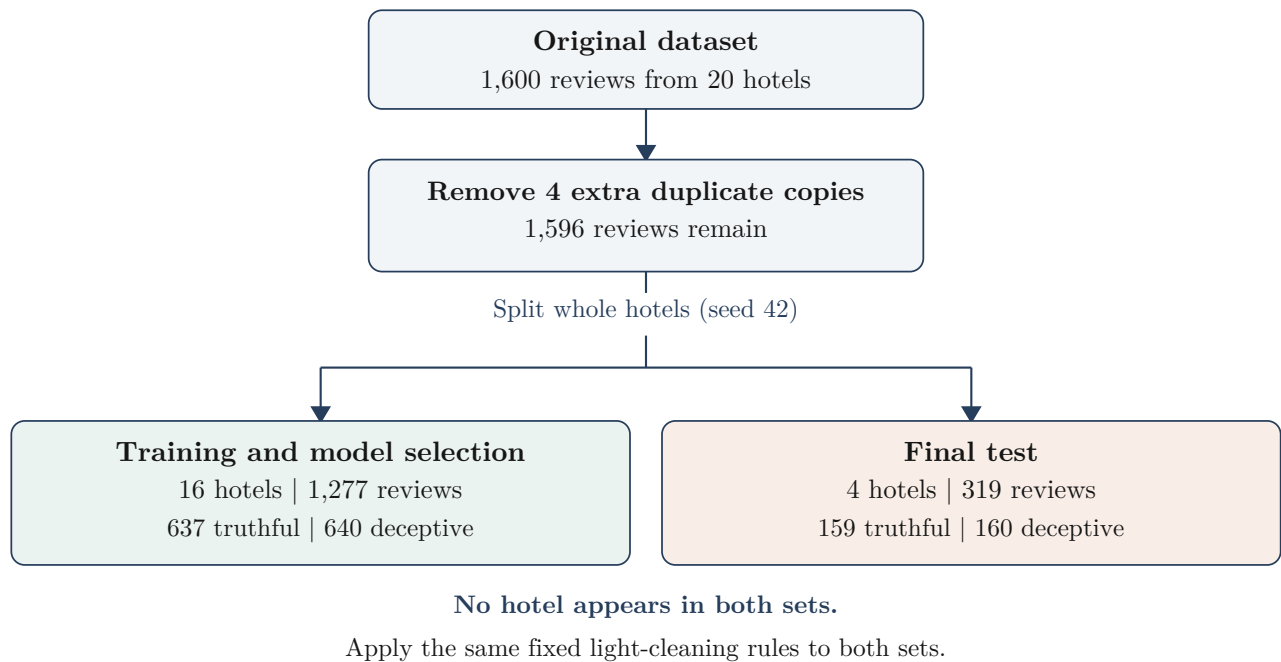


Figure 4. Duplicate removal and the hotel split, using counts recorded in Notebook 1. Entire hotels stay together; the same fixed cleaning rules are applied to both resulting sets.

Notebook 1 names the held-out portion `validation_df`. In the overall project, however, these 319 reviews serve as the final test set. Model comparison and hyperparameter selection use cross-validation within the 1,277 training reviews, as described in Section 3.

Applying light text cleaning

After splitting the hotels, I applied the same fixed cleaning function to both sets:

- Convert text to lowercase.
- Remove simple HTML tags while retaining the words inside them.
- Replace web addresses with the placeholder `<url>`.
- Collapse repeated whitespace and trim spaces at the beginning and end.

2.3 Cleaning, hotel split, and leakage prevention (continued)

These steps standardise formatting while keeping the review’s wording largely intact. HTML tags are removed before inserting `<url>`, so the placeholder is not mistaken for a tag. Contractions, numbers, punctuation, and spelling are preserved at this cleaning stage; their treatment during feature extraction is explained in Section 2.4.

The notebook records no HTML tags, one explicit URL, and repeated whitespace in 528 of the 1,596 reviews. After cleaning, there is one review containing `<url>`, no repeated whitespace, and no exact duplicate rows.

Preventing information from crossing the split

The cleaning rules are fixed transformations: they do not learn anything from either set. TF-IDF is different because it learns a vocabulary and word weights from the reviews supplied during fitting. These must therefore be learned from training reviews only, then applied to the held-out reviews.

The same boundary applies within cross-validation: each validation fold must be transformed using a vectoriser fitted only on that run’s training folds. The detailed feature construction and validation procedures follow in Sections 2.4 and 3.

Notebook 1 also checks that the training and test hotel sets are disjoint, contain 16 and four hotels respectively, and together account for all 1,596 retained reviews. These checks verify that the implemented split matches the intended design.

This preparation supports evaluation on unfamiliar hotels, but it does not remove the collection bias identified in Section 2.2. Source-related writing patterns can remain inside the text even when the source column is excluded.

2.4 Text preprocessing and TF-IDF feature construction

After preparing the dataset and separating the hotels, the next step was to turn each review into something the classifiers could work with. In Notebook 1, I used light text cleaning followed by term frequency-inverse document frequency, or TF-IDF. I prepared four representations to investigate two choices: whether neighbouring word pairs help, and whether reducing words to their base forms improves classification.

Light cleaning of review text

The cleaning procedure applies the same fixed rules to every review:

- Convert text to lowercase.
- Remove simple HTML tags while retaining the words inside them.
- Replace web addresses with the placeholder `<url>`.
- Collapse repeated whitespace and remove spaces at the beginning and end.

Lowercasing allows different capitalisations of a word to be treated consistently. Replacing URLs avoids retaining a particular address, while whitespace cleanup removes formatting differences. HTML tags are removed before inserting `<url>` so that the placeholder is preserved.

Among the 1,596 reviews remaining after duplicate removal, there were no HTML tags, one explicit URL, and repeated whitespace in 528 reviews. The implementation changed all 1,596 review texts, left no repeated whitespace, and introduced no exact duplicate rows.

The cleaning stage retains contractions, numbers, punctuation, and spelling. However, this does not mean that all these characters become model features: the later tokenisation stage extracts word-like tokens and excludes surrounding punctuation. This distinction matters because preserving the readable text and preserving information in the numerical representation are different things.

Converting words into TF-IDF weights

TF-IDF represents a review using numerical weights for terms in a learned vocabulary. A term can be an individual word or a pair of neighbouring words.

For a term t in review d , the implementation uses its occurrence count:

$$\text{tf}(t, d) = \text{number of occurrences of } t \text{ in } d$$

The inverse document frequency is:

$$\text{idf}(t) = \log \left(\frac{N + 1}{\text{df}(t) + 1} \right) + 1$$

Here, N is the number of training reviews, and $\text{df}(t)$ counts how many training reviews contain the term. The additional ones provide smoothing, implemented through `smooth_idf=True`.

The initial weight is then:

$$v_{t,d} = \text{tf}(t, d) \text{idf}(t)$$

2.4 Text preprocessing and TF-IDF feature construction (continued)

This gives terms that occur in fewer training reviews a higher IDF weight than terms appearing almost everywhere. It does not establish that a rare term indicates deception. TF-IDF constructs the features; the classifier subsequently learns how those features relate to the target labels.

Finally, `norm="l2"` normalises each nonzero review vector:

$$\mathbf{x}_d = \frac{\mathbf{v}_d}{\sqrt{\sum_t v_{t,d}^2}}$$

This reduces the influence of differences in vector magnitude caused by review length. It does not remove every relationship between length and the resulting features.

Individual words and neighbouring word pairs

The simplest representation uses unigrams, or individual tokens. For example, `not clean` contributes separate features for `not` and `clean`. This provides an interpretable starting point, but it does not explicitly preserve their relationship.

The second representation retains the unigrams and adds bigrams, which are pairs of neighbouring tokens. It can therefore include `not clean` as an additional feature, alongside its individual words. Other phrases, such as `front desk`, may also carry information that is less specific when the words are considered separately.

In the code, this choice corresponds to changing `ngram_range` from `(1, 1)` to `(1, 2)`. Bigrams preserve limited local word order, rather than complete sentence structure. They also produce a much larger feature space, and exact phrases can occur less consistently across reviews. Their usefulness therefore needs to be established through the later model comparisons.

Representations with and without lemmatisation

I also examined lemmatisation, which reduces suitable words to dictionary forms. For example, `rooms` can become `room`, allowing related forms to contribute to the same feature.

The implementation uses NLTK’s part-of-speech tagger and `WordNetLemmatizer`. The `wordnet_pos` function maps noun, verb, adjective, and adverb tags to WordNet’s categories. Supplying these tags allows verbs to be handled appropriately, instead of treating every token as a noun. Tokens without a supported category remain unchanged.

Table 6. Example lemmatisation transformations from Notebook 1

Original token	Lemma
rooms	room
was	be
were	be
had	have

The function preserves punctuation and spacing around the tokens it changes, and stores the lemmatised reviews separately from the original cleaned text. Its output can sound unnatural, such as “the staff be wonderful”, because it is an intermediate representation for feature extraction.

2.4 Text preprocessing and TF-IDF feature construction (continued)

Lemmatisation can combine related forms and reduce vocabulary size, but it can also remove distinctions in how a review was written. I therefore compared both versions rather than assuming that lemmatisation would improve prediction.

Shared vectoriser settings

The four representations use the same settings apart from their n-gram range and whether the text is lemmatised.

Table 7. Shared TF-IDF settings and their purposes

Setting	Implementation and purpose
Minimum document frequency	<code>min_df=2</code> retains terms appearing in at least two training reviews, reducing features based on isolated wording or typos.
Stop words	<code>stop_words=None</code> retains common words, including pronouns and negations that may contribute useful patterns.
Tokenisation	The custom token pattern retains one-letter tokens, numbers, and contractions using straight or curly apostrophes.
Lowercasing	<code>lowercase=False</code> , because lowercasing has already been performed during cleaning.
IDF smoothing	<code>smooth_idf=True</code> , matching the smoothed formula above.
Vector normalisation	<code>norm="l2"</code> scales each nonzero review vector to unit Euclidean length.

Document frequency counts reviews, rather than repetitions within a review. A word repeated ten times in one training review still fails `min_df=2`, whereas a word appearing once in each of two reviews is retained.

These shared settings were held fixed for the comparisons. Their inclusion is a design choice; the notebooks do not report a separate experiment establishing that each setting is optimal.

Resulting feature spaces and sparse storage

Fitting the four representations on the 1,277 training reviews produced the outputs shown in Table 8.

Table 8. Text feature counts and sparsity recorded in Notebook 1

Representation	Unigram features	Bigram features	Total text features	Nonzero features in first review
Unigrams, no lemmatisation	4,747	0	4,747	76
Unigrams and bigrams, no lemmatisation	4,747	18,851	23,598	144
Unigrams, with lemmatisation	3,908	0	3,908	75
Unigrams and bigrams, with lemmatisation	3,908	18,104	22,012	143

2.4 Text preprocessing and TF-IDF feature construction (continued)

Lemmatisation reduced the feature counts, while adding bigrams increased them substantially. These are properties of the representations, rather than evidence that one predicts deception better.

Each row represents one review, and each feature column corresponds to a learned term. Most entries are zero because an individual review contains only a small part of the vocabulary. The implementation therefore uses sparse matrices and pandas sparse dataframes.

The stored dataframes also retain four non-text fields for inspection. These additional columns are not predictive features: `deceptive` is separated as the target, and `hotel`, `source`, and `polarity` are excluded from model inputs.

The notebook checks that the training and held-out tables have identical columns in the same order and that feature rows align with their labels. These checks help prevent a classifier from receiving mismatched representations.

Learning the representation from training data

The distinction between `fit_transform` and `transform` is essential. `fit_transform` learns the vocabulary and IDF weights from the training reviews. `transform` applies that existing representation to held-out reviews, without changing its vocabulary or weights. Terms outside the learned vocabulary cannot introduce new feature columns.

Although Notebook 1 names the held-out tables “validation”, they represent the reviews reserved for the project’s final test. During model selection in Notebook 2, a new vectoriser is fitted within each cross-validation training fold. The tables fitted on all 1,277 training reviews are not reused to evaluate those folds.

A new review would follow the same cleaning and selected lemmatisation procedure, then be transformed using the fitted vectoriser before classification. This keeps its feature meanings and ordering consistent with those used to train the model.

3 Model Design and Cross-Validation

3.1 Shared pipeline, validation design, and metrics

With the data prepared, I could begin comparing models. I wanted to understand which combination of classifier and text representation would work most consistently on unfamiliar hotels. To make that comparison useful, every candidate followed the same evaluation procedure.

Notebook 2 uses the 1,277 training reviews from 16 hotels for model comparison. The 319 reviews from the four final test hotels remain outside model selection and hyperparameter tuning.

A shared prediction pipeline

I compared logistic regression, a linear SVM, and a small neural network across four text representations:

- Unigrams without lemmatisation.
- Unigrams and bigrams without lemmatisation.
- Lemmatised unigrams.
- Lemmatised unigrams and bigrams.

The linear models provide relatively simple classifiers for sparse text features. The neural network allows nonlinear combinations of those features, letting me investigate whether additional flexibility improves prediction. Their individual methods, training objectives, and settings are explained in Sections 3.2-3.4.

Each validation run creates a new scikit-learn Pipeline containing a TfidfVectorizer followed by the candidate classifier. The input is the prepared review text, with truthful/deceptive labels supplied during training. Hotel names determine the split but are excluded from prediction, alongside source and polarity.

The procedure within each run is:

1. Fit the vectoriser on the training folds, learning their vocabulary and IDF weights.
2. Fit the classifier using those training vectors and labels.
3. Transform the validation reviews using the already fitted vectoriser.
4. Predict their labels and calculate validation metrics, without refitting either component.

This distinction matters because fitting TF-IDF on all 16 hotels before cross-validation would allow the validation reviews to influence the vocabulary and weights. Notebook 2 therefore returns to the prepared text rather than using Notebook 1's already fitted TF-IDF tables.

The vectoriser settings remain consistent across model families. Terms must appear in at least two training reviews within the current run; common words, contractions, and one-character tokens are retained; smoothed IDF and L2 vector normalisation are used. The representation choices change whether bigrams and lemmatisation are included.

Five hotel-grouped validation folds

I used GroupKFold(`n_splits=5`), with hotel names supplied as the grouping variable. All reviews from a hotel stay together, so each run evaluates the model on hotels absent from that run's training data.

3.1 Shared pipeline, validation design, and metrics (continued)

Table 9. Hotel groups used for cross-validation (training N = 1,277)

Validation fold	Hotels	Reviews	Truthful	Deceptive
1	Fairmont, James, Talbott	240	120	120
2	Conrad, Intercontinental, Swissotel	240	120	120
3	Ambassador, Hyatt, Sofitel	240	120	120
4	Amalfi, Hilton, Sheraton	240	120	120
5	Affinia, Hardrock, Omni, Palmer	317	157	160
Total	16 hotels	1,277	637	640

Each run trains on four folds and validates on the remaining fold. This provides approximately 960-1,037 training reviews from 12-13 hotels per run. Across the five runs, every review receives one validation prediction for each candidate configuration.

The same split indices are reused for every classifier and representation. Before applying them to the lemmatised data, the notebook checks that its rows, labels, and metadata align with the non-lemmatised version. This makes comparisons refer to the same review groups.

The code also checks that training and validation hotels never overlap and that the five validation folds cover all training reviews exactly once.

Metrics and their practical meaning

The model predicts truthful = 0 or deceptive = 1. Here, the positive class means deceptive, regardless of whether the review expresses positive or negative sentiment.

For evaluation:

TP: a deceptive review correctly flagged.

FP: a truthful review incorrectly flagged.

FN: a deceptive review missed.

TN: a truthful review correctly classified.

Table 10. Validation metrics reported in Notebook 2

Metric	Definition	Meaning for ReviewBuster
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall proportion of correctly classified reviews.
Deceptive precision	$TP / (TP + FP)$	Among flagged reviews, how many carry the deceptive label?
Deceptive recall	$TP / (TP + FN)$	How many labelled deceptive reviews does the model detect?
Deceptive F1	$2TP / (2TP + FP + FN)$	Combines precision and recall for deceptive-review detection.
Macro F1	Mean of truthful-class F1 and deceptive-class F1	Gives both classes equal importance in the summary.

3.1 Shared pipeline, validation design, and metrics (continued)

I used mean deceptive F1 as the main comparison metric, supported by precision, recall, accuracy, and macro F1. Precision matters because incorrectly flagging truthful reviews could create unnecessary investigations. Recall matters because missed deceptive reviews would continue to escape attention. Macro F1 checks whether stronger deceptive-class performance comes at the expense of recognising truthful reviews.

For context, a rule that labels every review deceptive would achieve approximately 0.501 accuracy and 0.668 deceptive F1, averaged equally across these folds. These values are calculated directly from the label counts as an analytical reference; they are not results from an additional trained model. They show why deceptive F1 needs to be interpreted alongside the other metrics.

Training loss and the practical objective

The classifiers learn their parameters by minimising their training objectives. Logistic regression and the neural network use log-loss-based objectives, while the SVM uses squared hinge loss, together with regularisation. The evaluation functions then compare their predicted labels against the validation labels.

These training objectives do not directly optimise deceptive F1 or represent the real cost of investigating a truthful review versus missing a deceptive one. F1 provides a useful comparison criterion, but it does not specify those business costs either. Reporting precision and recall separately makes the two types of error visible.

Summarising the five runs

For each candidate, the notebook records individual fold results and reports the mean and sample standard deviation across five folds. The mean gives an overall comparison, while the standard deviation describes how much performance changes between hotel groups.

Each fold receives equal weight in these summaries, although Fold 5 contains more reviews. The reported mean is therefore an average of five fold scores, rather than a metric calculated from all validation predictions pooled together.

Each run also fits a separate model; the five fitted models are not combined into an ensemble. Their scores guide pipeline selection, after which the chosen pipeline is trained on the complete training portion.

Because these validation results are used to choose the pipeline, they are not its final test performance. Fold variation is also descriptive, rather than proof that a small difference between candidates is statistically significant. The separate test hotels provide the later assessment of the selected pipeline.

3.2 Logistic regression: method and results

Model choice and learning objective

I used logistic regression as the first model for evaluating whether the linguistic features extracted from hotel reviews could distinguish truthful reviews from deceptive ones. It provides a relatively simple starting point: each TF-IDF feature receives a learned coefficient, and these coefficients collectively determine the model’s prediction.

This suits the project’s sparse text representation, where a review contains only a small proportion of the thousands of possible vocabulary features. Logistic regression also provides a useful reference for assessing whether the more complex models evaluated later offer a meaningful improvement.

For a review represented by the TF-IDF vector $\mathbf{x}$, the model computes:

$$z = \mathbf{w}^T \mathbf{x} + b, \quad p(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-z}}$$

Here, $\mathbf{w}$ contains the learned feature coefficients, b is the intercept, and class 1 represents a deceptive review. A positive coefficient increases the deceptive-class score when its feature is present; a negative coefficient reduces it. These associations are learned from the dataset and should not be interpreted as proof that particular words indicate dishonesty.

The classifier learns its parameters by minimising logistic loss with an L2 regularisation penalty. For one labelled review, the logistic loss is:

$$\ell(y, p) = -y \log(p) - (1 - y) \log(1 - p)$$

This penalises predictions that assign low probability to the correct label, particularly confident incorrect predictions. The notebook implements this process through `LogisticRegression.fit()` within a Pipeline; the loss and optimisation procedure are handled by the library. Predictions use the default probability boundary of 0.5. See the [scikit-learn logistic regression guide](#).

Experimental settings and parameter rationale

The experiment compared four text representations: non-lemmatised unigrams; non-lemmatised unigrams and bigrams; lemmatised unigrams; and lemmatised unigrams and bigrams.

Unigrams test how much information individual words provide. Adding bigrams tests whether short phrases improve discrimination by preserving some local word order. Lemmatisation tests whether combining related word forms improves generalisation or removes distinctions that are useful for this task.

Each representation was evaluated with $C = 0.1, 1$, and 10 , giving 12 configurations. Here, C controls the inverse strength of regularisation: smaller values apply stronger shrinkage to the model’s coefficients.

3.2 Logistic regression: method and results (continued)

Table 11. Logistic regression parameters and task-specific rationale

Parameter	Setting and reason
penalty="l2"	Shrinks large coefficients while allowing many vocabulary features to contribute. This is a reasonable constraint when the feature count substantially exceeds the number of training reviews and the goal is prediction on unfamiliar hotels. L1 was not evaluated, so L2 is a justified starting choice rather than a proven optimal penalty.
C = 0.1	Tests stronger regularisation, reducing reliance on individual training features. This checks whether greater constraint improves generalisation across hotels.
C = 1	Provides the library's standard starting value and a middle point between the stronger and weaker settings. Its default status does not establish that it is optimal for ReviewBuster.
C = 10	Tests weaker regularisation, allowing stronger coefficients when the training evidence supports them. This checks whether the more constrained settings suppress useful distinctions.
solver="liblinear"	Supports binary classification, the L2 penalty, and sparse TF-IDF inputs. It provides a suitable numerical optimisation method for this modest number of reviews. Alternative solvers were not compared.
fit_intercept=True	Allows a learned baseline score in addition to the vocabulary contributions, avoiding a requirement that the decision boundary pass through the feature-space origin.
class_weight=None	Gives each training example equal weight. There are 637 truthful and 640 deceptive training reviews, so class-frequency imbalance does not require correction. Different business costs for the two error types would need a separate justification for weighting.
max_iter=1000	Gives the solver more opportunity to converge than the default iteration limit. It controls optimisation effort rather than the model's representational capacity.
random_state=42	Fixes the seed for reproducibility where the solver uses randomness. The particular number is not intended to improve predictive performance.

The liblinear implementation also regularises the intercept through its added constant feature. These parameter behaviours are documented in the [scikit-learn LogisticRegression](#) reference.

The TF-IDF settings described in Section 2.4 were held constant to make the comparisons easier to interpret. Terms had to appear in at least two training reviews (`min_df=2`), limiting features supported by only one document. Stop words were retained because function words and negation may contribute to review-writing patterns. The token pattern preserved contractions and single-character words, while additional lowercasing was disabled because it had already been performed during cleaning. Smoothed IDF and L2 vector normalisation were applied consistently.

L2 vector normalisation controls the scale of each review's input features; L2 regularisation constrains the classifier's learned coefficients.

3.2 Logistic regression: method and results (continued)

Validation procedure

All configurations used the same five hotel-grouped validation splits drawn from the 1,277 training reviews. Reviews from a hotel appeared entirely in either the training portion or the validation portion of a run. This evaluates the intended behaviour of predicting reviews from hotels that the model has not encountered during training.

A fresh TF-IDF vectoriser and classifier were fitted within every fold. Consequently, the vocabulary, document frequencies, and classifier coefficients were learned using only that fold’s training reviews. The 319 final test reviews remained outside these experiments.

The 12 configurations required 60 pipeline fits. Their scores were summarised using the mean and sample standard deviation across the five folds.

Deceptive-class F1 was the principal comparison metric because it combines precision and recall for the reviews ReviewBuster aims to flag. Precision measures how often a flagged review is labelled deceptive; recall measures how many labelled deceptive reviews are detected. Accuracy and macro F1 provide supporting information about overall performance and both classes.

Results across representations and regularisation settings

Tables 12-15 separate the four text representations so that changes in regularisation can be compared within each setting. All values are mean $\pm$ sample standard deviation across the same five hotel-grouped folds. Precision, recall, and deceptive F1 refer to class 1.

Non-lemmatised text

Table 12. Non-lemmatised text: unigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.802 $\pm$ 0.015	0.807 $\pm$ 0.029	0.797 $\pm$ 0.056	0.800 $\pm$ 0.022	0.801 $\pm$ 0.015
1	0.839 $\pm$ 0.031	0.851 $\pm$ 0.032	0.825 $\pm$ 0.063	0.836 $\pm$ 0.037	0.839 $\pm$ 0.031
10	0.860 $\pm$ 0.034	0.877 $\pm$ 0.032	0.838 $\pm$ 0.061	0.857 $\pm$ 0.040	0.860 $\pm$ 0.035

Table 13. Non-lemmatised text: unigram and bigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.819 $\pm$ 0.017	0.827 $\pm$ 0.035	0.814 $\pm$ 0.065	0.818 $\pm$ 0.024	0.819 $\pm$ 0.018
1	0.851 $\pm$ 0.024	0.855 $\pm$ 0.032	0.847 $\pm$ 0.060	0.850 $\pm$ 0.029	0.851 $\pm$ 0.024
10	0.879 $\pm$ 0.029	0.888 $\pm$ 0.030	0.868 $\pm$ 0.058	0.877 $\pm$ 0.033	0.879 $\pm$ 0.029

3.2 Logistic regression: method and results (continued)

Lemmatised text

Table 14. Lemmatised text: unigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.804 ± 0.023	0.803 ± 0.035	0.809 ± 0.060	0.804 ± 0.028	0.803 ± 0.023
1	0.840 ± 0.022	0.848 ± 0.040	0.832 ± 0.048	0.839 ± 0.024	0.840 ± 0.022
10	0.857 ± 0.031	0.868 ± 0.046	0.845 ± 0.060	0.855 ± 0.035	0.856 ± 0.032

Table 15. Lemmatised text: unigram and bigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.825 ± 0.027	0.824 ± 0.045	0.831 ± 0.057	0.826 ± 0.029	0.824 ± 0.028
1	0.851 ± 0.027	0.862 ± 0.046	0.840 ± 0.048	0.850 ± 0.028	0.851 ± 0.027
10	0.868 ± 0.033	0.877 ± 0.041	0.859 ± 0.058	0.867 ± 0.036	0.868 ± 0.033

Effect of regularisation

Mean deceptive F1 increased as C increased in all four representations. For non-lemmatised unigrams and bigrams, it rose from 0.818 at C=0.1, to 0.850 at C=1, and to 0.877 at C=10. This suggests that stronger regularisation restricted useful learning in these experiments. However, training scores were not reported alongside these validation results, so this pattern alone cannot establish underfitting.

C=10 was the best value tested, but it was also the upper boundary of the search. The experiment therefore does not establish the globally optimal regularisation strength.

Effect of bigrams

Adding bigrams improved mean deceptive F1 at every tested C for both preprocessing variants. At C=10, non-lemmatised F1 increased from 0.857 to 0.877, while lemmatised F1 increased from 0.855 to 0.867. These results support retaining short phrase information alongside individual words. They do not identify which phrases caused the improvement; that would require examining the learned coefficients and individual predictions.

Effect of lemmatisation

Lemmatisation produced mixed results. It slightly improved unigram F1 at C=0.1 and C=1, but slightly reduced it at C=10. With bigrams, it improved F1 under stronger regularisation, produced the same rounded mean at C=1, and reduced F1 from 0.877 to 0.867 at C=10.

Combining word forms can reduce vocabulary fragmentation, but it can also remove distinctions in wording that the classifier finds useful. This is a possible explanation for the mixed results, rather than a mechanism demonstrated by the experiment. No statistical significance test was performed.

3.2 Logistic regression: method and results (continued)

Strongest configuration and practical interpretation

The strongest logistic regression configuration used non-lemmatised unigrams and bigrams with $C=10$.

Table 16. Strongest logistic regression configuration

Metric	Mean $\pm$ standard deviation
Accuracy	0.879 ± 0.029
Deceptive precision	0.888 ± 0.030
Deceptive recall	0.868 ± 0.058
Deceptive F1	0.877 ± 0.033
Macro F1	0.879 ± 0.029

Its precision exceeded its recall, indicating that it was more successful at keeping flagged reviews relevant than at detecting every deceptive review. Both errors matter for the project: a false positive could cause unnecessary suspicion of a truthful customer, while a false negative could leave a deceptive review unflagged.

Performance also varied across hotel groups. For this configuration, Fold 3 - containing Ambassador, Hyatt, and Sofitel - had a deceptive F1 of 0.823 and recall of 0.775. The fold contained 120 reviews from each class, so its lower performance cannot be attributed to class imbalance alone. Examining its misclassified reviews would be necessary to explain the difficulty.

Limitations and conclusion

Logistic loss provides a tractable training objective, but it does not directly optimise deceptive F1 or the operational consequences of incorrect flags. A model can improve its probability estimates without changing its predicted labels, and therefore without improving F1. The project addresses this mismatch by selecting configurations using hotel-grouped validation metrics and examining precision and recall separately. Business-specific error costs and threshold tuning were not evaluated.

The dataset’s source-label relationship remains another limitation. Excluding source metadata prevents it from becoming a direct predictor, but source-related writing patterns may still appear in the review text. These results consequently demonstrate performance across held-out hotels within this corpus; they do not establish reliable detection of deception across all review platforms or hospitality settings.

Overall, logistic regression provides a competitive reference model. Non-lemmatised unigrams and bigrams with $C=10$ achieved its strongest observed validation performance and is carried forward into the comparison with SVM and MLP in Section 3.5.

3.3 Linear SVM: method and results

Why I tried a linear SVM

After logistic regression, I wanted to see whether a different way of learning the boundary between truthful and deceptive reviews would improve the predictions. I therefore tested a linear support vector machine using scikit-learn's LinearSVC.

The model still assigns weights to the TF-IDF features, but its training objective focuses on separating the two classes with a margin. This provides a useful comparison with logistic regression while keeping the review representations and validation folds consistent.

A linear model is a sensible starting point for these sparse text features. Adding bigrams already gives it access to short phrases, so it can use information beyond individual words without introducing a nonlinear kernel. Nonlinear kernels were not evaluated in these experiments.

How the model makes a prediction

For a review represented by the TF-IDF vector x , the SVM calculates a decision score:

$$s(x) = w^T x + b$$

Here, w contains the learned feature weights and b is the intercept. Positive scores lead to deceptive predictions, while negative scores lead to truthful predictions. The decision boundary is at zero.

Unlike logistic regression, LinearSVC does not directly produce probabilities. Its score indicates which side of the boundary a review falls on and its position relative to that boundary. A score of 0.8 therefore cannot be interpreted as an 80% probability that a review is deceptive. See the scikit-learn SVM guide.

Learning objective and optimisation

The notebook uses L2 regularisation with squared hinge loss. To express this objective, let $t_i=+1$ represent a deceptive review and $t_i=-1$ a truthful review. This is mathematical notation; the notebook continues to pass the original 0 and 1 labels to the classifier.

The loss for a review is:

$$\ell_i = [\max(0, 1 - t_i s(x_i))]^2$$

The quantity $t_i s(x_i)$ is the signed margin. A review receives zero loss when it is correctly classified with a signed margin of at least one. Reviews inside the margin, including some correctly classified reviews, still receive a penalty. Misclassified reviews receive a larger penalty as they move further onto the wrong side.

With the default intercept scaling of one, the unweighted training objective is:

$$J(w, b) = \frac{1}{2}(\|w\|_2^2 + b^2) + C \sum_{i=1}^n \ell_i$$

This balances smaller coefficients against margin violations. The intercept is also regularised through LIBLINEAR's added constant feature. Smaller C values strengthen regularisation relative to the loss. See LIBLINEAR's formulation.

3.3 Linear SVM: method and results (continued)

The notebook sets `dual="auto"`. Because these fold-specific vocabularies contain more features than training reviews, the supported L2 squared-hinge configuration selects the dual formulation. LIBLINEAR solves this using coordinate descent, updating individual dual variables associated with training examples. The library performs these updates when `pipeline.fit()` is called. See the [LinearSVC](#) reference.

Settings and experimental procedure

The same four text representations were evaluated at $C=0.1$, 1, and 10. The remaining SVM parameters were held constant so that the results could be compared within each representation.

Table 17. Linear SVM parameters and task-specific rationale

Parameter	Setting and reason
<code>penalty="l2"</code>	Constrains large coefficients while allowing many vocabulary features to contribute.
<code>loss="squared_hinge"</code>	Penalises margin violations, with greater emphasis on larger violations.
<code>C = 0.1, 1, 10</code>	Tests stronger, intermediate and weaker regularisation.
<code>dual="auto"</code>	Selects a compatible optimisation formulation based on the sample and feature counts.
<code>fit_intercept=True</code>	Allows a baseline term alongside the word and phrase contributions.
<code>class_weight=None</code>	Applies equal example weights, consistent with the nearly balanced training labels: 637 truthful and 640 deceptive reviews.
<code>max_iter=5000</code>	Allows additional optimisation iterations.
<code>random_state=42</code>	Controls randomness in the dual solver.

Squared hinge loss was held fixed during these trials. Its quadratic penalty can make large violations more influential than ordinary hinge loss, which matters if some reviews are unusual or incorrectly labelled. The notebook did not compare the two losses, so squared hinge should be described as the implemented choice rather than an established winner.

The four text representations and three C values produced 12 configurations and 60 pipeline fits. Every configuration used the five hotel-grouped folds described in Section 3.1. A fresh vectoriser and classifier were fitted within each fold, keeping validation reviews outside vocabulary construction and model training. The 319 final test reviews were excluded from this comparison.

The saved SVM runs contain no convergence warnings, but the notebook does not record iteration counts for each fit. The iteration limit alone therefore does not demonstrate convergence.

Validation results

Tables 18-21 separate non-lemmatised and lemmatised text. Within each group, unigrams and unigrams plus bigrams have their own table, comparing $C=0.1$, 1, and 10 across all five metrics. Values are mean $\pm$ sample standard deviation across the same five hotel-grouped folds. Precision, recall, and deceptive F1 refer to class 1.

3.3 Linear SVM: method and results (continued)

Non-lemmatised text

Table 18. Non-lemmatised text: unigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.836 ± 0.033	0.848 ± 0.033	0.820 ± 0.067	0.833 ± 0.040	0.836 ± 0.034
1	0.865 ± 0.038	0.884 ± 0.036	0.840 ± 0.072	0.860 ± 0.045	0.864 ± 0.038
10	0.855 ± 0.037	0.875 ± 0.032	0.830 ± 0.072	0.851 ± 0.045	0.855 ± 0.038

Table 19. Non-lemmatised text: unigram and bigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.851 ± 0.024	0.855 ± 0.032	0.847 ± 0.060	0.850 ± 0.029	0.851 ± 0.024
1	0.884 ± 0.031	0.898 ± 0.030	0.869 ± 0.060	0.882 ± 0.036	0.884 ± 0.031
10	0.886 ± 0.028	0.898 ± 0.027	0.872 ± 0.055	0.884 ± 0.031	0.886 ± 0.028

Lemmatised text

Table 20. Lemmatised text: unigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.839 ± 0.021	0.848 ± 0.039	0.830 ± 0.048	0.838 ± 0.023	0.839 ± 0.021
1	0.861 ± 0.033	0.877 ± 0.040	0.842 ± 0.062	0.858 ± 0.038	0.861 ± 0.033
10	0.854 ± 0.042	0.868 ± 0.040	0.838 ± 0.079	0.851 ± 0.049	0.854 ± 0.042

Table 21. Lemmatised text: unigram and bigram results

C	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.1	0.850 ± 0.028	0.860 ± 0.047	0.839 ± 0.048	0.848 ± 0.029	0.849 ± 0.028
1	0.876 ± 0.022	0.883 ± 0.033	0.868 ± 0.040	0.875 ± 0.024	0.876 ± 0.022
10	0.874 ± 0.024	0.882 ± 0.030	0.865 ± 0.046	0.873 ± 0.026	0.874 ± 0.024

3.3 Linear SVM: method and results (continued)

Strongest tested configuration

The strongest tested SVM configuration used non-lemmatised unigrams and bigrams with $C=10$.

Table 22. Strongest observed linear SVM configuration

Metric	Mean $\pm$ standard deviation
Accuracy	0.886 ± 0.028
Deceptive precision	0.898 ± 0.027
Deceptive recall	0.872 ± 0.055
Deceptive F1	0.884 ± 0.031
Macro F1	0.886 ± 0.028

What changed the performance?

Adding bigrams improved the five-fold mean of every reported metric at each C value, with both preprocessing choices. For non-lemmatised text at $C=10$, deceptive-class F1 increased from 0.851 to 0.884. This supports including neighbouring word combinations alongside individual words.

The regularisation results were less straightforward. Moving from $C=0.1$ to $C=1$ improved performance across all four representations. Increasing C again to 10 reduced unigram performance and slightly reduced both lemmatised results. Only non-lemmatised unigrams and bigrams improved, from 0.882 to 0.884 deceptive-class F1.

That gain is small relative to the variation across folds. It gives $C=10$ the highest observed mean in this grid, but does not establish a statistically reliable advantage over $C=1$. It also shows why weaker regularisation should not automatically be expected to improve predictions.

Lemmatisation provided no consistent benefit. With unigrams and bigrams, it lowered mean deceptive-class F1 at all three C values. At $C=10$, the score fell from 0.884 to 0.873. These results give no predictive reason to add lemmatisation to this SVM pipeline.

Fold 3 remained difficult across the tested configurations. For the strongest SVM, its macro F1 was 0.837, compared with 0.891-0.905 in the other folds, and deceptive-class recall was 0.783. This fold contains the Ambassador, Hyatt and Sofitel hotels and has equal numbers of truthful and deceptive reviews. Class imbalance therefore does not explain its lower performance; inspecting individual errors would be needed to understand the cause.

3.3 Linear SVM: method and results (continued)

Connecting the loss to the project's goal

Squared hinge loss rewards correct predictions with sufficient separation from the boundary. F1 measures the resulting class decisions. For example, a deceptive review with a signed margin of 0.5 is correctly classified but still receives a loss of 0.25. Increasing its margin beyond one removes that loss without changing its predicted label or contribution to F1.

This mismatch is tolerated because the margin objective provides a practical way to learn a regularised boundary. Deceptive-class F1, precision and recall then monitor whether that boundary produces useful decisions on unfamiliar hotels.

For the strongest configuration, precision exceeds recall: the model is relatively accurate when flagging a review, but still misses some labelled deceptive reviews. Any later threshold adjustment would need to consider both missed reviews and false accusations against genuine customers, using training and validation data.

Overall, non-lemmatised unigrams and bigrams with $C=10$ is the strongest SVM configuration tested in Notebook 2. These validation results support taking it forward to the model comparison and subsequent refinement stages.

3.4 MLP: method and results

Model choice and architecture

I included a multilayer perceptron (MLP) to investigate whether nonlinear combinations of text features could improve deceptive-review detection. Logistic regression and the linear SVM produce a linear score from the TF-IDF inputs. The MLP introduces a hidden layer that can learn combinations whose effects depend on which other features are present.

The network used one hidden layer containing 16 ReLU units, followed by one sigmoid output unit for binary classification:

TF-IDF review vector $\longrightarrow$ **16 ReLU units** $\longrightarrow$ **Sigmoid probability** $\longrightarrow$ **Predicted class**

For a review vector $\mathbf{x}$, its forward computation can be written as:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1), \quad \text{ReLU}(a) = \max(0, a)$$

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}_2^\top \mathbf{h} + b_2)$$

Here, $\mathbf{W}_1$ connects the TF-IDF features to the hidden units, $\mathbf{b}_1$ contains their biases, and $\mathbf{w}_2$ and b_2 produce the output score. The sigmoid converts this score into the estimated probability of the deceptive class. ReLU introduces the nonlinearity that distinguishes this network from a single linear classifier.

In the implementation, `hidden_layer_sizes=(16,)` defines the hidden layer, while `activation="relu"` specifies its activation function. The binary output activation is handled by `MLPClassifier`.

Although the hidden layer contains only 16 units, the network still has many parameters because each unit connects to every input feature. With D TF-IDF features, it has $16D + 33$ trainable weights and biases. For example, the first non-lemmatised unigram-and-bigram fold contained 19,671 features, giving 314,769 parameters. This helps explain why controlling model flexibility matters with approximately one thousand training reviews per fold.

Training objective and optimisation

The MLP was trained using binary cross-entropy with an L2 penalty on its weight matrices:

$$J = \frac{1}{n} \sum_{i=1}^n [-y_i \log(p_i) - (1 - y_i) \log(1 - p_i)] + \frac{\alpha}{2n} (\|\mathbf{W}_1\|_F^2 + \|\mathbf{w}_2\|_2^2)$$

Here, n is the number of training reviews in a fold, y_i is the label of review i , and p_i is its predicted deceptive probability. The first term rewards probabilities that agree with the training labels. The second penalises large weights. In this implementation, the penalty applies to the weights rather than the biases.

Backpropagation computes how changes in the parameters affect this objective. At the sigmoid output, the error term is $p_i - y_i$; this error is propagated through the output weights and the ReLU derivative to calculate the hidden-layer gradients. `MLPClassifier.fit()` handles these calculations internally, and the L-BFGS solver uses the resulting loss and gradients to update the parameters. See the `scikit-learn` MLP implementation.

3.4 MLP: method and results (continued)

Experimental settings and parameter rationale

The experiment retained the four text representations used for the linear models: non-lemmatised unigrams; non-lemmatised unigrams and bigrams; lemmatised unigrams; and lemmatised unigrams and bigrams.

Each representation was evaluated with $\alpha=0.01, 0.1, 1$, and 10 . This produced 16 configurations and 80 pipeline fits across the five hotel-grouped folds.

Unlike C in logistic regression and SVM, larger α means stronger regularisation. Their numerical values are not directly equivalent across model families.

Table 23. MLP parameters and task-specific rationale

Parameter	Setting and reason
<code>hidden_layer_sizes=(16,)</code>	Introduces capacity to combine features while limiting hidden-layer width for the available training data. Sixteen units is a starting architecture; the main comparison does not establish it as optimal.
<code>activation="relu"</code>	Allows nonlinear responses to combinations of vocabulary features, directly testing whether additional flexibility helps this task.
<code>solver="lbfgs"</code>	Uses a full-batch quasi-Newton optimiser. It is a reasonable choice for the modest number of training reviews per fold. Its superiority over Adam or SGD was not tested here.
<code>alpha = 1</code>	Provides the experiment's baseline penalty, allowing stronger and weaker regularisation to be compared. This was an experimental starting value, not the library default or an established optimum.
<code>alpha = 10</code>	Tests whether stronger shrinkage reduces reliance on patterns specific to the training hotels.
<code>alpha = 0.1</code>	Tests whether relaxing the baseline penalty allows useful text relationships to be learned more effectively.
<code>alpha = 0.01</code>	Further relaxes the penalty to investigate whether additional flexibility improves validation performance.
<code>max_iter=300</code>	Limits L-BFGS optimisation iterations while allowing more iterations than the library default. These are solver iterations rather than minibatch training epochs.
<code>max_fun=15000</code>	Limits loss-function evaluations, providing an additional bound on optimisation effort. An iteration can require multiple evaluations.
<code>tol=1e-4</code>	Retains a consistent convergence tolerance across trials. With this solver, it is passed to L-BFGS as its gradient tolerance. A tighter tolerance was not evaluated.
<code>random_state=42</code>	Makes initial weights and biases reproducible for each fixed input configuration. A single seed does not establish robustness to different initialisations.

3.4 MLP: method and results (continued)

L-BFGS does not use minibatches, so batch size and the stochastic optimisers' learning-rate settings do not control these trials. The notebook also does not use an internal early-stopping validation split. See the `scikit-learn` `MLPClassifier` reference.

The TF-IDF settings remained consistent with Sections 2.4 and 3.2: `min_df=2`, no stop-word removal, the contraction-preserving token pattern, existing lowercasing, smoothed IDF, and L2 vector normalisation. Holding these settings constant helps isolate the effects of the classifier, lemmatisation, n-gram representation, and regularisation strength.

Validation procedure

Every configuration used the same five hotel-grouped splits within the 1,277 training reviews. Each run trained on 960 or 1,037 reviews and predicted reviews belonging to hotels excluded from that run's training data.

A fresh pipeline fitted its TF-IDF vocabulary, IDF weights, and network parameters using only the training portion. The corresponding validation reviews were transformed and classified without refitting either component. The 319 final test reviews remained outside these experiments.

The notebook reported accuracy, deceptive precision, deceptive recall, deceptive F1, and macro F1. Deceptive F1 was the principal selection metric, while precision and recall made the two error types visible. Results below are mean $\pm$ sample standard deviation across five folds.

Results: Non-lemmatised text

Table 24. Non-lemmatised text: unigram results

Alpha	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.01	0.855 $\pm$ 0.030	0.874 $\pm$ 0.025	0.830 $\pm$ 0.057	0.851 $\pm$ 0.036	0.855 $\pm$ 0.030
0.1	0.864 $\pm$ 0.032	0.883 $\pm$ 0.031	0.840 $\pm$ 0.063	0.860 $\pm$ 0.038	0.863 $\pm$ 0.033
1	0.864 $\pm$ 0.035	0.885 $\pm$ 0.035	0.837 $\pm$ 0.064	0.859 $\pm$ 0.041	0.863 $\pm$ 0.035
10	0.851 $\pm$ 0.036	0.864 $\pm$ 0.036	0.835 $\pm$ 0.069	0.848 $\pm$ 0.043	0.851 $\pm$ 0.037

Table 25. Non-lemmatised text: unigram and bigram results

Alpha	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.01	0.884 $\pm$ 0.032	0.899 $\pm$ 0.027	0.866 $\pm$ 0.055	0.882 $\pm$ 0.036	0.884 $\pm$ 0.032
0.1	0.884 $\pm$ 0.028	0.901 $\pm$ 0.024	0.864 $\pm$ 0.053	0.881 $\pm$ 0.032	0.884 $\pm$ 0.028
1	0.883 $\pm$ 0.031	0.896 $\pm$ 0.028	0.867 $\pm$ 0.060	0.880 $\pm$ 0.036	0.883 $\pm$ 0.031
10	0.717 $\pm$ 0.200	0.610 $\pm$ 0.374	0.728 $\pm$ 0.411	0.652 $\pm$ 0.375	0.650 $\pm$ 0.290

Adding bigrams improved all five mean metrics at `alpha=0.01`, `0.1`, and `1`. At `alpha=0.01`, deceptive F1 increased from 0.851 to 0.882. This supports including adjacent phrases alongside individual words.

However, progressively reducing regularisation did not produce a large improvement. Combined-representation F1 was 0.880, 0.881, and 0.882 at `alpha=1`, `0.1`, and `0.01`, respectively. These small differences do not establish a meaningful advantage for the weakest penalty.

3.4 MLP: method and results (continued)

Results: Lemmatised text

Table 26. Lemmatised text: unigram results

Alpha	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.01	0.851 $\pm$ 0.040	0.863 $\pm$ 0.044	0.835 $\pm$ 0.057	0.848 $\pm$ 0.043	0.851 $\pm$ 0.040
0.1	0.854 $\pm$ 0.038	0.869 $\pm$ 0.035	0.835 $\pm$ 0.076	0.850 $\pm$ 0.046	0.853 $\pm$ 0.039
1	0.855 $\pm$ 0.039	0.873 $\pm$ 0.038	0.832 $\pm$ 0.074	0.850 $\pm$ 0.047	0.854 $\pm$ 0.040
10	0.855 $\pm$ 0.023	0.864 $\pm$ 0.042	0.846 $\pm$ 0.058	0.853 $\pm$ 0.028	0.854 $\pm$ 0.024

Table 27. Lemmatised text: unigram and bigram results

Alpha	Accuracy	Precision	Recall	Deceptive F1	Macro F1
0.01	0.868 $\pm$ 0.030	0.879 $\pm$ 0.030	0.854 $\pm$ 0.061	0.865 $\pm$ 0.035	0.868 $\pm$ 0.031
0.1	0.874 $\pm$ 0.025	0.884 $\pm$ 0.029	0.864 $\pm$ 0.052	0.873 $\pm$ 0.029	0.874 $\pm$ 0.026
1	0.872 $\pm$ 0.026	0.879 $\pm$ 0.036	0.864 $\pm$ 0.047	0.870 $\pm$ 0.028	0.871 $\pm$ 0.026
10	0.859 $\pm$ 0.031	0.871 $\pm$ 0.047	0.847 $\pm$ 0.052	0.857 $\pm$ 0.032	0.859 $\pm$ 0.031

For lemmatised text, adding bigrams improved every mean metric at all four penalty settings. The strongest lemmatised configuration used alpha=0.1, reaching 0.873 deceptive F1.

Regularisation again showed no steadily improving trend. Combined-representation F1 peaked at alpha=0.1, then fell to 0.865 at alpha=0.01. More flexible weights therefore did not automatically generalise better.

Lemmatisation generally reduced performance relative to the stronger non-lemmatised combined configurations. For example, at alpha=0.01, deceptive F1 fell from 0.882 to 0.865. The exception at alpha=10 needs separate interpretation because its non-lemmatised counterpart suffered two failed folds.

Failure under strong regularisation

The non-lemmatised unigram-and-bigram configuration at alpha=10 produced constant class predictions in two folds:

Table 28. Failed folds for non-lemmatised unigrams and bigrams at alpha=10

Fold	Prediction behaviour	Accuracy	Deceptive F1	Macro F1
3	Every review predicted deceptive	0.500	0.667	0.333
5	Every review predicted truthful	0.495	0.000	0.331

Fold 5 produced no deceptive predictions, making deceptive precision undefined. Scikit-learn issued an UndefinedMetricWarning and recorded precision as 0.0 in the results.

3.4 MLP: method and results (continued)

The notebook’s discussion reports that a diagnostic rerun found almost-zero weights and nearly constant deceptive probabilities: approximately 0.5015 in Fold 3 and 0.4999 in Fold 5. These values fall on opposite sides of the classification boundary, explaining the different constant labels.

Strong regularisation interacting with the inputs and optimisation is a plausible explanation. However, the experiments do not isolate why these two folds failed while the others did not. The lemmatised configuration avoided this collapse, but that single comparison does not establish a general benefit from lemmatisation.

This failure also demonstrates why accuracy and deceptive F1 should be interpreted alongside macro F1 and prediction behaviour. Predicting every balanced-fold review as deceptive still produces deceptive F1 of 0.667, despite providing no discrimination between the classes.

Strongest configuration and practical interpretation

The strongest observed MLP configuration used non-lemmatised unigrams and bigrams with $\alpha=0.01$.

Table 29. Strongest observed MLP configuration

Metric	Mean $\pm$ standard deviation
Accuracy	0.884 ± 0.032
Deceptive precision	0.899 ± 0.027
Deceptive recall	0.866 ± 0.055
Deceptive F1	0.882 ± 0.036
Macro F1	0.884 ± 0.032

Precision exceeded recall, indicating that flagged reviews were relatively often labelled deceptive, while some deceptive reviews remained undetected.

Fold 3, containing Ambassador, Hyatt, and Sofitel, remained difficult even for this stronger configuration. Its deceptive F1 was 0.819 and recall was 0.775. Its balanced labels indicate that class imbalance alone does not explain the lower performance.

The selected MLP’s mean deceptive F1 was close to the linear SVM’s 0.884. This small observed difference does not demonstrate that the MLP’s nonlinear architecture improves detection. Its additional flexibility also introduces more parameters and sensitivity to training initialisation.

3.4 MLP: method and results (continued)

Limitations and conclusion

Cross-entropy is useful for learning probabilities, but it does not directly optimise deceptive F1 or the consequences of incorrect flags. A false positive could place unnecessary suspicion on a truthful review; a false negative could allow a deceptive review to remain unflagged. Validation precision, recall, and F1 monitor these outcomes, but business-specific error costs and threshold tuning were not evaluated.

MLP training has a non-convex objective, so different initialisations can produce different solutions. Fixing the seed supports reproducibility, while repeated-seed experiments would be needed to assess this source of variation. See the [scikit-learn neural network guide](#).

The stored outputs contain no MLP convergence warnings, but the evaluation functions do not retain iteration counts or final training losses for every run. Their absence therefore does not establish that every model reached a satisfactory solution. Systematic optimisation diagnostics and training-versus-validation comparisons would strengthen the analysis.

The source-label relationship discussed earlier also remains relevant: a network can learn collection-source writing patterns from the text even when source metadata is excluded.

Overall, the MLP achieved competitive validation performance with non-lemmatised unigrams and bigrams. The $\alpha=0.01$ configuration is carried forward into Section 3.5 as its strongest tested setting. The results support evaluating nonlinear models, while providing no clear evidence that this architecture offers an advantage over the linear SVM.

3.5 Comparison across models and text representations

Purpose and basis of comparison

After evaluating the models individually, I compared their results to understand which choices contributed most to deceptive-review detection. The comparison addresses three questions:

- Does adding bigrams improve prediction over unigrams alone?
- Does lemmatisation improve prediction over the original cleaned text?
- Which model family performs best with the selected text representation?

Each comparison uses the same five hotel-grouped validation folds. Vocabulary construction, IDF estimation, and classifier training remain confined to each run’s training portion. This keeps the comparison aligned with the project’s goal of predicting reviews from unfamiliar hotels.

The complete experiments covered 40 configurations and 200 pipeline fits:

Table 30. Scope of the model and representation comparisons

Model	Regularisation values	Text representations	Configurations	Pipeline fits
Logistic regression	C=0.1, 1, 10	4	12	60
Linear SVM	C=0.1, 1, 10	4	12	60
MLP	alpha=0.01, 0.1, 1, 10	4	16	80

Deceptive-class F1 is the principal comparison metric. Precision, recall, accuracy, and macro F1 provide supporting evidence. All means give equal weight to the five folds; they are not metrics calculated from pooled predictions across all validation reviews. This distinction matters because four folds contain 240 reviews each, while the fifth contains 317.

Comparing unigrams with unigrams and bigrams

To assess bigrams, I paired results from the same model, regularisation setting, preprocessing choice, and held-out fold. The only changed component was the inclusion of adjacent two-word terms.

The tables report mean deceptive F1 for each representation. The difference column gives the mean $\pm$ sample standard deviation of the five paired fold differences.

3.5 Comparison across models and text representations (continued)

Table 31. Effect of adding bigrams to non-lemmatised text

Model	Setting	Unigram F1	Unigram + bigram F1	Paired change in F1	Improved folds
Logistic regression	C=0.1	0.800 $\pm$ 0.022	0.818 $\pm$ 0.024	+0.018 $\pm$ 0.007	5/5
Logistic regression	C=1	0.836 $\pm$ 0.037	0.850 $\pm$ 0.029	+0.014 $\pm$ 0.012	5/5
Logistic regression	C=10	0.857 $\pm$ 0.040	0.877 $\pm$ 0.033	+0.021 $\pm$ 0.013	5/5
Linear SVM	C=0.1	0.833 $\pm$ 0.040	0.850 $\pm$ 0.029	+0.017 $\pm$ 0.015	5/5
Linear SVM	C=1	0.860 $\pm$ 0.045	0.882 $\pm$ 0.036	+0.022 $\pm$ 0.014	5/5
Linear SVM	C=10	0.851 $\pm$ 0.045	0.884 $\pm$ 0.031	+0.034 $\pm$ 0.018	5/5
MLP	alpha=0.01	0.851 $\pm$ 0.036	0.882 $\pm$ 0.036	+0.031 $\pm$ 0.005	5/5
MLP	alpha=0.1	0.860 $\pm$ 0.038	0.881 $\pm$ 0.032	+0.022 $\pm$ 0.008	5/5
MLP	alpha=1	0.859 $\pm$ 0.041	0.880 $\pm$ 0.036	+0.021 $\pm$ 0.011	5/5
MLP	alpha=10	0.848 $\pm$ 0.043	0.652 $\pm$ 0.375	-0.196 $\pm$ 0.389	3/5

Table 32. Effect of adding bigrams to lemmatised text

Model	Setting	Unigram F1	Unigram + bigram F1	Paired change in F1	Improved folds
Logistic regression	C=0.1	0.804 $\pm$ 0.028	0.826 $\pm$ 0.029	+0.021 $\pm$ 0.022	4/5
Logistic regression	C=1	0.839 $\pm$ 0.024	0.850 $\pm$ 0.028	+0.011 $\pm$ 0.006	5/5
Logistic regression	C=10	0.855 $\pm$ 0.035	0.867 $\pm$ 0.036	+0.012 $\pm$ 0.014	4/5
Linear SVM	C=0.1	0.838 $\pm$ 0.023	0.848 $\pm$ 0.029	+0.010 $\pm$ 0.008	5/5
Linear SVM	C=1	0.858 $\pm$ 0.038	0.875 $\pm$ 0.024	+0.017 $\pm$ 0.018	4/5
Linear SVM	C=10	0.851 $\pm$ 0.049	0.873 $\pm$ 0.026	+0.022 $\pm$ 0.027	5/5
MLP	alpha=0.01	0.848 $\pm$ 0.043	0.865 $\pm$ 0.035	+0.017 $\pm$ 0.013	5/5
MLP	alpha=0.1	0.850 $\pm$ 0.046	0.873 $\pm$ 0.029	+0.023 $\pm$ 0.020	5/5
MLP	alpha=1	0.850 $\pm$ 0.047	0.870 $\pm$ 0.028	+0.020 $\pm$ 0.023	5/5
MLP	alpha=10	0.853 $\pm$ 0.028	0.857 $\pm$ 0.032	+0.004 $\pm$ 0.014	3/5

Differences were calculated before rounding. Consequently, subtracting the displayed means may occasionally produce a slightly different last decimal place.

3.5 Comparison across models and text representations (continued)

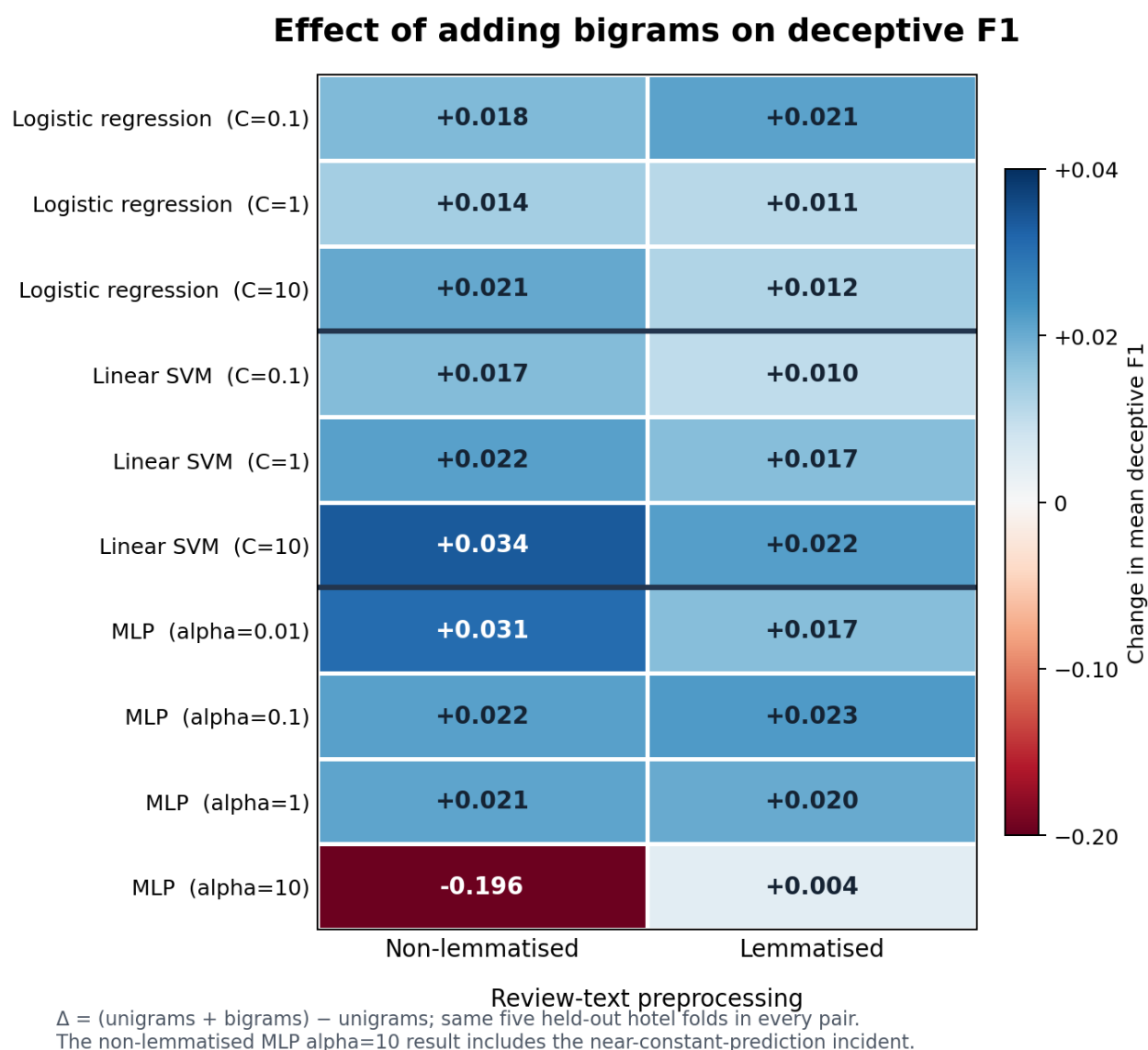


Figure 5. Change in mean deceptive F1 after adding bigrams, extracted from Notebook 2. Each pair uses the same model setting and hotel folds.

Adding bigrams improved mean deceptive F1 in 19 of the 20 matched settings, and in 93 of the 100 setting-by-fold comparisons. These comparisons reuse the same reviews, so the counts describe consistency rather than independent statistical tests.

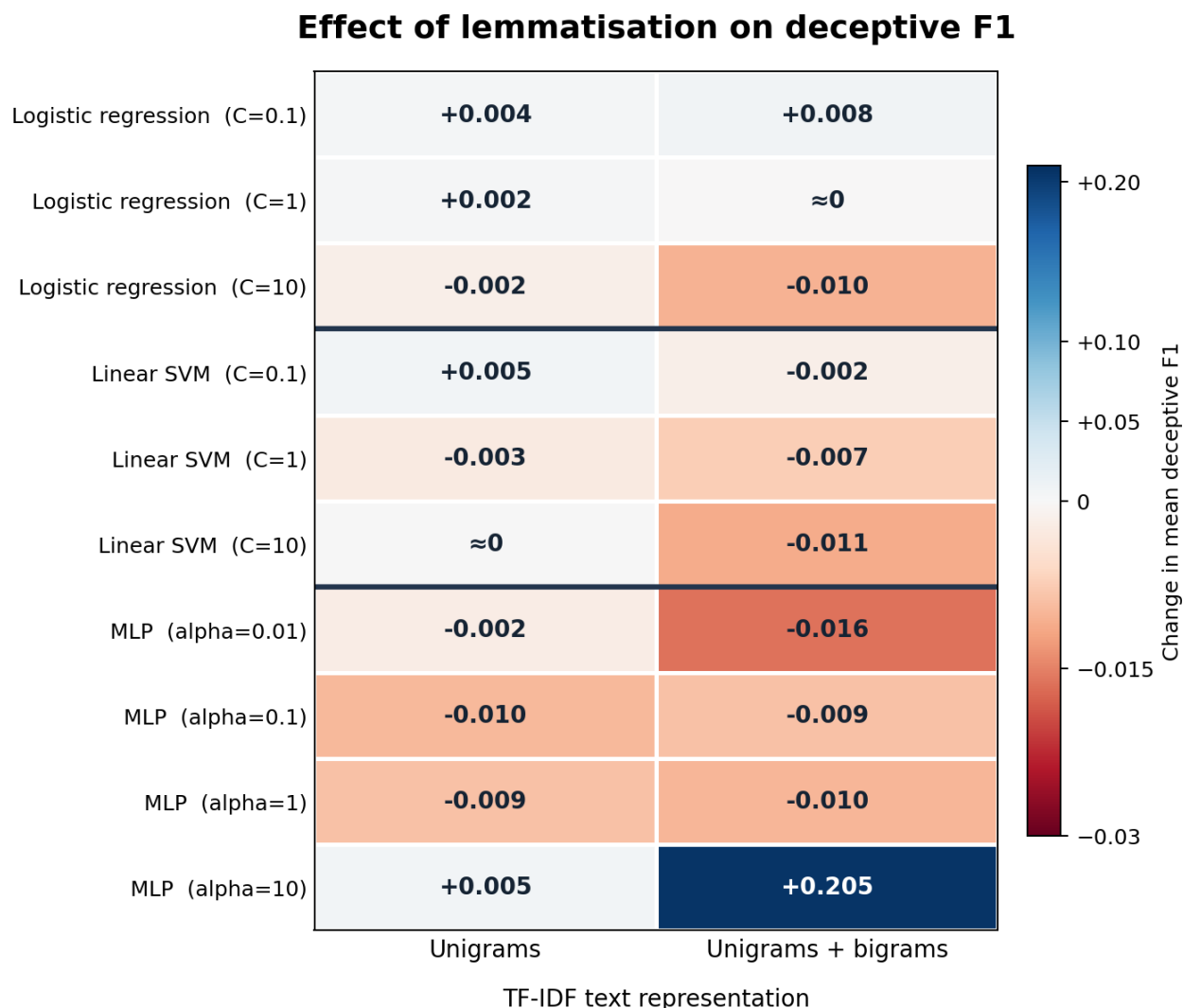
The non-lemmatised SVM at C=10 provides particularly relevant evidence: bigrams increased mean deceptive F1 by 0.034 and macro F1 by 0.031, with deceptive F1 improving in all five folds. This supports retaining short phrases alongside individual words.

The exception was the non-lemmatised MLP at alpha=10. Its combined representation produced constant class predictions in two folds, causing the large negative average difference. The other three folds still improved after adding bigrams. This result identifies a problematic model setting rather than demonstrating that bigrams are generally harmful.

3.5 Comparison across models and text representations (continued)

Comparing lemmatised and non-lemmatised text

The preprocessing comparison holds the model, regularisation setting, representation, and validation fold fixed while changing whether the text is lemmatised.



Δ = lemmatised – non-lemmatised; each cell uses matching folds and model settings.

The MLP alpha=10 combined case compares against a non-lemmatised run with near-constant predictions.

Figure 6. Change in mean deceptive F1 after lemmatisation, extracted from Notebook 2. Positive values favour lemmatisation; approximately zero denotes a magnitude below 0.0005.

The following tables summarise the changes across each family's tested regularisation settings. The median describes the typical setting-level change, while the range preserves the variation between settings.

3.5 Comparison across models and text representations (continued)

Table 33. Lemmatisation effects with unigrams

Model	Median change in F1	Setting-level change range	Settings favouring lemmatisation	Fold comparisons favouring lemmatisation
Logistic regression	+0.002	-0.002 to +0.004	2/3	7/15
Linear SVM	~0	-0.003 to +0.005	2/3	6/15
MLP	-0.006	-0.010 to +0.005	1/4	8/20

Table 34. Lemmatisation effects with unigrams and bigrams

Model	Median change in F1	Setting-level change range	Settings favouring lemmatisation	Fold comparisons favouring lemmatisation
Logistic regression	~0	-0.010 to +0.008	1/3	7/15
Linear SVM	-0.007	-0.011 to -0.002	0/3	6/15
MLP	-0.009	-0.016 to +0.205	1/4	7/20

Change means lemmatised minus non-lemmatised. The notation ~0 denotes a magnitude below 0.0005.

Lemmatisation improved mean deceptive F1 in 7 of 20 matched settings and 41 of 100 setting-by-fold comparisons. Its benefits were therefore inconsistent, particularly when bigrams were included.

For the combined-representation SVM, lemmatisation reduced mean deceptive F1 at all three tested C values. At C=10, performance fell from 0.884 to 0.873. This provides no predictive reason to add lemmatisation to that pipeline.

The MLP’s large positive change at alpha=10 requires caution. Its lemmatised version avoided the two failed folds observed without lemmatisation. The difference reflects that particular failure and should not be interpreted as a general improvement from reducing words to their base forms.

Overall, the comparisons favour non-lemmatised unigrams and bigrams for the subsequent model-family comparison.

Comparing the strongest tested model settings

With the text input fixed to non-lemmatised unigrams and bigrams, I compared each family’s strongest tested setting for mean deceptive F1:

- Logistic regression at C=10.
- Linear SVM at C=10.
- The 16-unit, single-hidden-layer MLP at alpha=0.01.

3.5 Comparison across models and text representations (continued)

Table 35. Validation metrics for the selected setting from each family

Metric	Logistic regression	Linear SVM	MLP
Accuracy	0.879 ± 0.029	0.886 ± 0.028	0.884 ± 0.032
Deceptive precision	0.888 ± 0.030	0.898 ± 0.027	0.899 ± 0.027
Deceptive recall	0.868 ± 0.058	0.872 ± 0.055	0.866 ± 0.055
Deceptive F1	0.877 ± 0.033	0.884 ± 0.031	0.882 ± 0.036
Macro F1	0.879 ± 0.029	0.886 ± 0.028	0.884 ± 0.032

Values are mean $\pm$ sample standard deviation across five hotel-grouped folds. Bold entries identify the highest displayed mean.

The SVM achieved the highest mean accuracy, deceptive recall, deceptive F1, and macro F1. The MLP’s precision was higher by 0.001, while its deceptive F1 was lower by 0.002. Logistic regression remained competitive, trailing the SVM by approximately 0.007 deceptive F1.

These are small observed differences. The fold standard deviations describe variation between hotel groups; they are not confidence intervals or a significance test. No formal statistical comparison establishes that one family is reliably superior.

Performance across individual hotel folds

Figure 7 and Table 36 compare deceptive F1 for the selected model settings on the same held-out hotel groups.

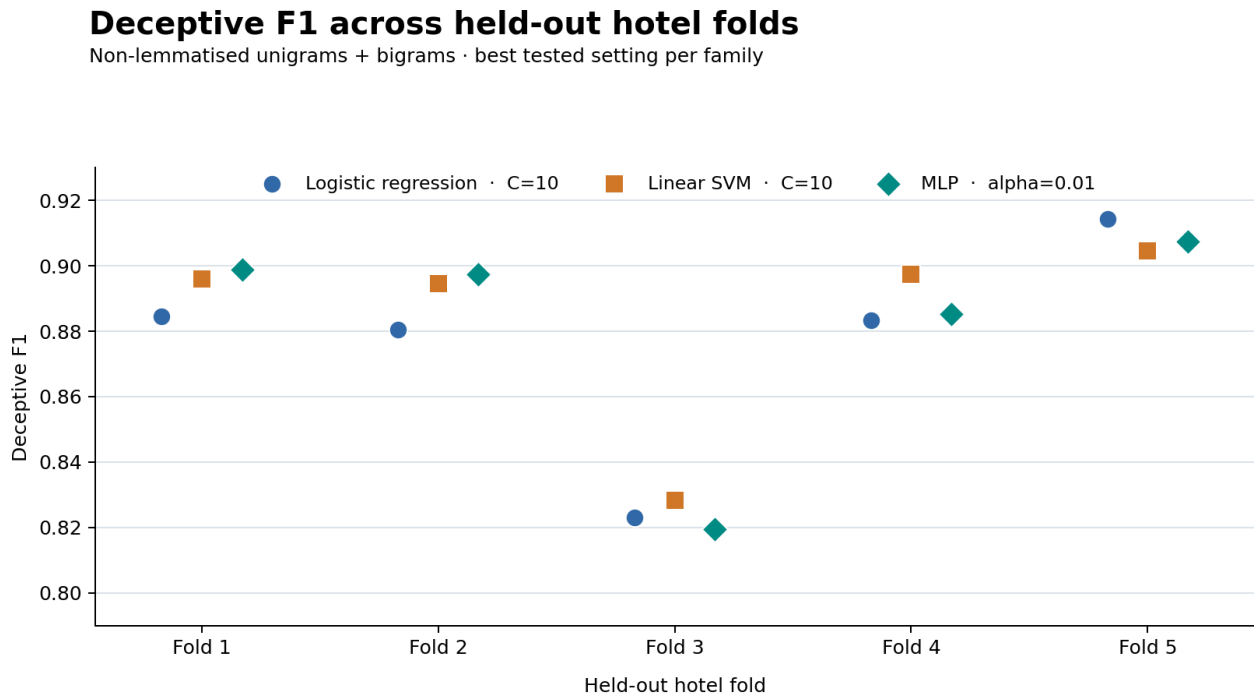


Figure 7. Deceptive F1 for the selected model settings on the same held-out hotel folds, extracted from Notebook 2. The vertical axis is restricted to approximately 0.79-0.93 to show the differences clearly.

3.5 Comparison across models and text representations (continued)

Table 36. Deceptive F1 by validation fold

Fold	Logistic regression	Linear SVM	MLP
1	0.884	0.896	0.899
2	0.880	0.895	0.897
3	0.823	0.828	0.819
4	0.883	0.897	0.885
5	0.914	0.904	0.907

No model led on every fold. MLP performed best on Folds 1 and 2, SVM on Folds 3 and 4, and logistic regression on Fold 5.

Fold 3 was the most difficult group for all three selected configurations. It contains Ambassador, Hyatt, and Sofitel, with 120 truthful and 120 deceptive reviews. Its lower performance cannot therefore be attributed to class imbalance alone.

Deceptive recall in this fold was 0.775 for logistic regression, 0.783 for SVM, and 0.775 for MLP. All three missed more deceptive reviews here than their overall recall summaries might suggest. The shared difficulty indicates that examining this group’s individual errors would be more informative than attributing the result solely to one model’s architecture.

Connecting the comparison to the project’s objective

The models optimise different training objectives. Logistic regression and MLP minimise cross-entropy with regularisation, while SVM minimises squared hinge loss with regularisation. Their numerical training losses therefore cannot be used directly to rank the families. The shared validation metrics compare the class decisions produced by those different objectives.

Deceptive F1 balances precision and recall, but it does not assign explicit business costs to errors. False positives could place suspicion on truthful customers, while false negatives could leave deceptive reviews unflagged. The selected configurations all have higher precision than recall, so useful flagging performance coexists with missed deceptive reviews.

The MLP’s nonlinear architecture did not deliver a clear improvement over the linear SVM. Its additional architectural and initialisation choices therefore have no demonstrated performance benefit in this comparison. Runtime and memory usage were not benchmarked, so the results do not establish an efficiency ranking.

3.5 Comparison across models and text representations (continued)

Limitations and comparison summary

The same validation folds were used to choose settings and report their comparative performance. Selecting the highest-scoring configurations can make their reported validation results optimistic. The separate 319-review final test addresses this concern later by evaluating the refined pipeline on hotels excluded from model selection.

The comparisons also involve only five hotel groups, and their training sets overlap. Fold counts and repeated comparisons should therefore remain descriptive. Repeated-seed MLP experiments, systematic training diagnostics, and individual error analysis would strengthen the evidence.

Finally, hotel grouping does not remove the dataset’s source-label relationship. The models may still learn source-related writing patterns, limiting how confidently these results can be extended to other review platforms.

Taken together, the experiments support three conclusions: bigrams generally improve prediction; lemmatisation provides no consistent benefit; and the linear SVM has the strongest observed mean performance among the selected configurations. These findings provide the evidence for the primary pipeline selection in Section 3.6.

3.6 Primary pipeline selection

Following the comparisons in Section 3.5, I selected a linear SVM using non-lemmatised TF-IDF unigrams and bigrams as ReviewBuster’s primary pipeline.

This choice brings together three findings from Notebook 2. Adding bigrams generally improved prediction, suggesting that short phrases contributed useful information alongside individual words. Lemmatisation provided no consistent benefit and reduced deceptive-class F1 in all three matched SVM settings using bigrams. The linear SVM also achieved the highest mean deceptive-class F1 among the strongest tested configurations.

Selected pipeline

Table 37. Selected primary pipeline and decision rationale

Component	Selection	Basis for the decision
Review preprocessing	Lightly cleaned text without lemmatisation	The matched validation results gave no predictive reason to add lemmatisation.
Text representation	TF-IDF unigrams and bigrams, <code>ngram_range=(1, 2)</code>	Adding bigrams improved deceptive-class F1 in all five folds for the selected SVM setting.
Feature construction	Retain the settings described in Section 2.4	Keeps the selected pipeline consistent with the evaluated experiments, including <code>min_df=2</code> , retained stop words and L2 vector normalisation.
Classifier	LinearSVC with L2 regularisation and squared hinge loss	Produced the strongest observed mean results while retaining a simpler structure than the tested MLP.
Initial regularisation setting	<code>C=10</code>	Highest mean deceptive-class F1 among the SVM configurations tested in Notebook 2; carried forward as the reference for further refinement.
Main selection criterion	Mean deceptive-class F1 across five hotel-grouped folds	Balances precision and recall when predicting reviews from hotels excluded from each run’s training data.

At this setting, the SVM achieved deceptive-class F1 of 0.884 ± 0.031 and macro F1 of 0.886 ± 0.028 . The corresponding deceptive-class F1 scores were 0.882 ± 0.036 for the MLP and 0.877 ± 0.033 for logistic regression.

The differences were small, and no model led on every hotel fold. I therefore treated the SVM as the strongest observed candidate rather than claiming that it was conclusively superior. Its selection also reflects the absence of a demonstrated benefit from the MLP’s additional nonlinear structure. Runtime and memory usage were not benchmarked, so efficiency was not an established advantage.

Moving from selection to final evaluation

The selected pipeline is a configuration to carry forward; the five cross-validation models are not combined into the final classifier. Section 4 describes further refinement of C using training data, followed by fitting a fresh vectoriser and classifier on all 1,277 training reviews from 16 hotels.

The resulting pipeline is then evaluated on the 319 reviews from four reserved test hotels. This keeps the final evaluation separate from the validation results used for selection. Those results assess prediction against the dataset’s recorded labels; they do not establish whether an individual reviewer was dishonest.

4 Hyperparameter Refinement and Final Test Evaluation

4.1 SVM hyperparameter tuning

Section 3 identified $C=10$ as the strongest tested setting for the selected SVM. However, this was also the upper limit of the original search. I therefore investigated additional values to identify a promising interval, then examined that interval using smaller increments.

Both stages used the same five hotel-grouped validation folds and selected candidates primarily by mean deceptive F1. The reserved test reviews remained outside the tuning process.

Stage 1: Exploring a broader range

The first stage tested $C = 1, 3, 5, 10, 20, 30$ and 50 .

These relatively large steps provided an initial view of how performance changed across a broader range. Values below 10 examined whether stronger regularisation could maintain the previous result, while values up to 50 checked whether increasing C offered further improvement.

Table 38. Broad-search results from Notebook 3

C	Deceptive F1	Macro F1
1	0.882 ± 0.036	0.884 ± 0.031
3	0.882 ± 0.035	0.884 ± 0.031
5	0.883 ± 0.035	0.886 ± 0.031
10	0.884 ± 0.031	0.886 ± 0.028
20	0.884 ± 0.031	0.886 ± 0.028
30	0.883 ± 0.031	0.885 ± 0.027
50	0.883 ± 0.031	0.885 ± 0.027

Values are mean $\pm$ sample standard deviation across five hotel-grouped folds.

The strongest results occurred at $C=10$ and $C=20$, which shared an unrounded mean deceptive F1 of 0.884121. Increasing C further to 30 or 50 slightly reduced performance.

This suggested that progressively increasing C was unlikely to provide a useful improvement within the tested range. I retained $C=10$ as the reference because it matched $C=20$ while applying stronger regularisation. The next stage examined the surrounding interval more closely, particularly values below 10.

4.1 SVM hyperparameter tuning (continued)

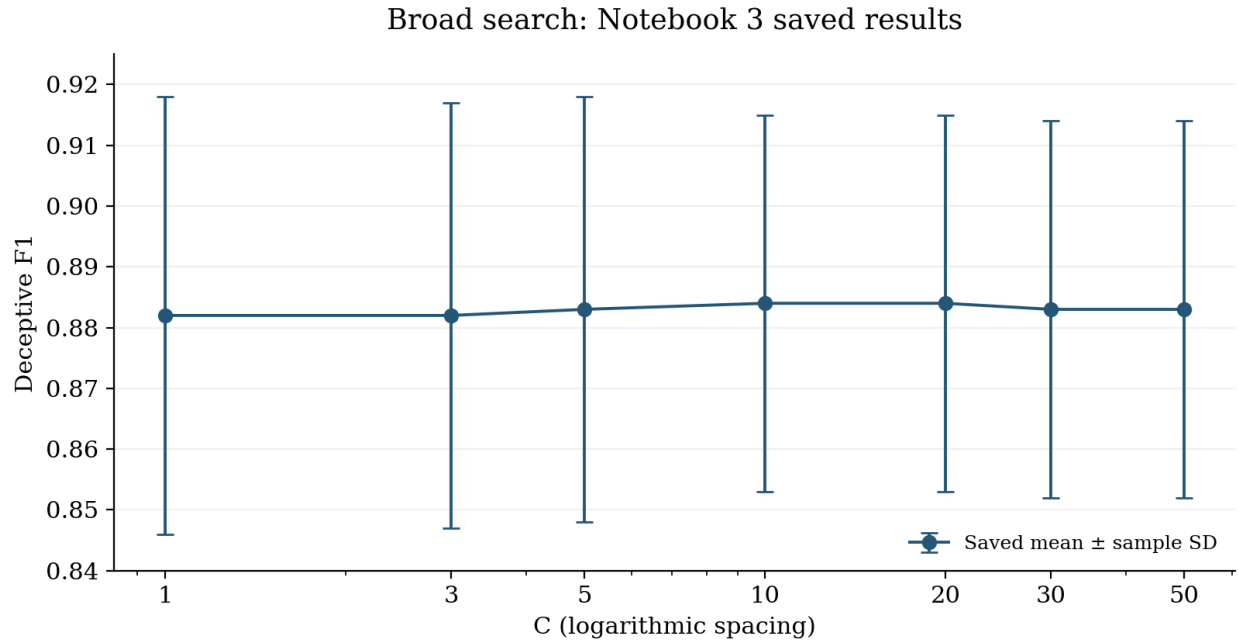


Figure 8. Broad-search results plotted from Notebook 3’s saved table. Error bars show sample standard deviation across folds.

Stage 2: Refining the interval with increments of 0.1

The second stage tested $C=5.0$ to $C=15.0$, in increments of 0.1, producing 101 candidates.

This finer search examined the region around $C=10$ that the broader steps could not describe in detail. It allowed me to investigate whether a nearby value could improve the validation result while retaining stronger regularisation.

Table 39. Representative results from the fine search

C	Deceptive F1	Macro F1
5.0	0.883 ± 0.035	0.886 ± 0.031
5.5	0.884 ± 0.034	0.886 ± 0.030
6.0	0.886 ± 0.030	0.888 ± 0.027
6.1	0.886 ± 0.030	0.888 ± 0.027
7.0	0.885 ± 0.032	0.887 ± 0.028
10.0	0.884 ± 0.031	0.886 ± 0.028
15.0	0.884 ± 0.031	0.886 ± 0.028

Values are mean $\pm$ sample standard deviation across five hotel-grouped folds.

4.1 SVM hyperparameter tuning (continued)

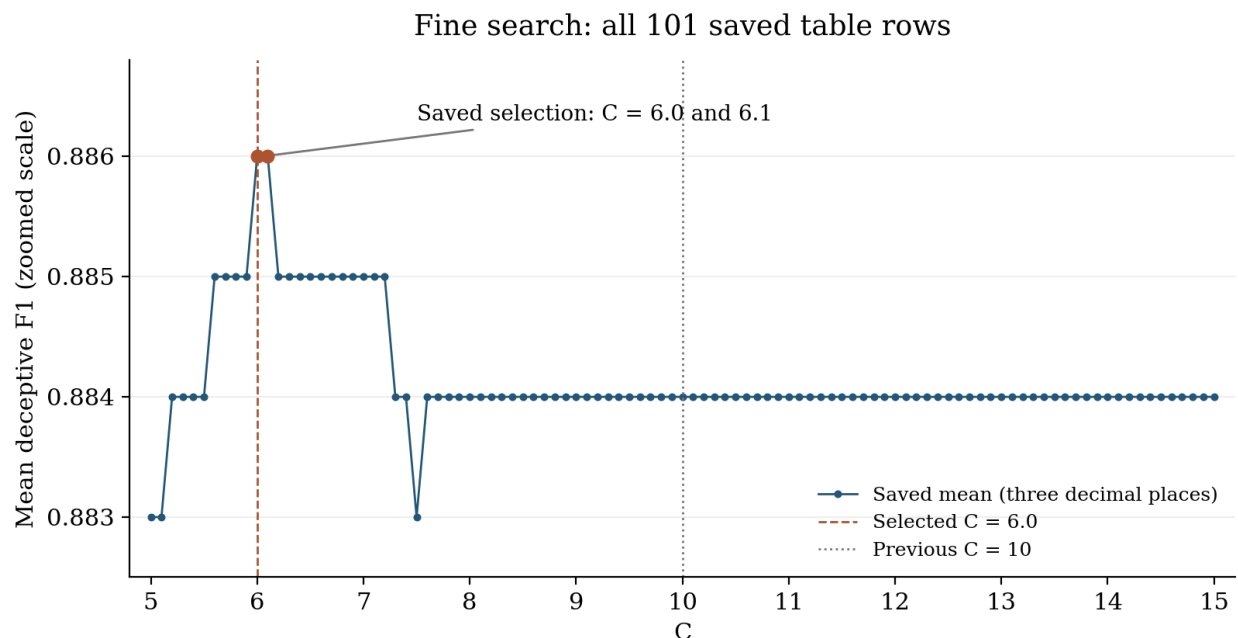


Figure 9. All 101 candidates plotted from the saved table’s three-decimal means. The vertical axis is zoomed to show the small differences; selection used unrounded scores.

The highest unrounded mean deceptive F1 was 0.885919, reached at both $C=6.0$ and $C=6.1$. Their mean macro F1 was 0.887622. Following the notebook’s tie rule, I selected $C=6.0$ for slightly stronger regularisation.

The improvement over $C=10$ was approximately 0.0018, much smaller than the fold-to-fold standard deviation of approximately 0.030. The result therefore supports $C=6.0$ as the strongest setting observed in this search, without establishing a reliable advantage on future hotels.

The notebook also records only eight distinct unrounded mean deceptive-F1 scores across the 101 candidates. This indicates that many neighbouring values produced the same measured performance. The finer search helped identify a favourable region, but its small increments should not be interpreted as locating an exact optimum.

Outcome of the refinement

The two stages progressively narrowed the search: the broad comparison found no benefit from extending C beyond 20, while the finer comparison identified $C=6.0$ as the selected setting.

This value was fixed before final testing. The following section describes final training and evaluation using that configuration.

4.2 Final training and held-out test procedure

Fixing the configuration before testing

After the refinement in Section 4.1, I fixed the final model as `LinearSVC(C=6.0)` using non-lemmatised TF-IDF unigrams and bigrams. The remaining classifier settings stayed consistent with the earlier experiments: L2 regularisation, squared hinge loss, `dual="auto"`, a fitted intercept, no class weighting, `random_state=42`, and `max_iter=5000`.

At this point, the model-selection stage was complete. The reserved test reviews would assess the chosen configuration, and their results would not be used to change the model, preprocessing or decision threshold.

Preparing the final inputs

Notebook 3 loads the unigram-and-bigram feature dataframes prepared in Notebook 1:

```
unigram_bigram_training_df.pkl.gz  
unigram_bigram_validation_df.pkl.gz
```

Despite its filename, the second dataframe contains the reserved final test set. It is separate from the five validation folds used for model selection.

Table 40. Final training and held-out test inputs

Dataset	Reviews	Hotels	Truthful reviews	Deceptive reviews	TF-IDF features
Final training	1,277	16	637	640	23,598
Held-out test	319	4	159	160	23,598

The TF-IDF vectoriser had learned its vocabulary and IDF weights from all 1,277 training reviews in Notebook 1. It then transformed the reserved reviews using that fitted representation. Notebook 3 reuses these numerical features; it does not fit another vectoriser on the test set.

This gives both datasets the same 23,598 feature columns. A column must represent the same word or phrase in the same position during training and prediction, so Notebook 3 checks that their feature names and ordering match exactly.

The notebook also checks the expected review and hotel counts, confirms that the training and test hotels are disjoint, and verifies that the target labels contain the expected 0 and 1 values.

Only columns beginning with `tfidf_` enter the classifier. The deceptive column supplies the target labels, while hotel, source and polarity remain available for analysis. These metadata fields are excluded from the predictive inputs.

The selected feature columns are converted from pandas sparse dataframes into compressed sparse row matrices:

$$X_{\text{train}} \in \mathbb{R}^{1277 \times 23598}, \quad X_{\text{test}} \in \mathbb{R}^{319 \times 23598}$$

This preserves the sparse representation instead of creating a dense matrix containing mostly zeros.

Training one final model

During cross-validation, each model learned from four folds and predicted the fifth. Once the configuration was selected, those validation folds had completed their role. I therefore fitted one new SVM using all 1,277 training reviews and their labels:

```
final_svm.fit(X_final_train, y_final_train)
```

4.2 Final training and held-out test procedure (continued)

This learns one final coefficient vector and intercept using the full training dataset. The five cross-validation classifiers are not averaged or combined into an ensemble.

The final fit uses the same regularised squared-hinge objective explained in Section 3.3. Notebook 3 also converts any `ConvergenceWarning` into an error during fitting. If the solver reports non-convergence, execution stops before test evaluation. The saved run completed this guarded fitting step and produced the evaluation outputs.

Predicting the held-out reviews

The fitted classifier then receives the test feature matrix:

```
test_predictions = final_svm.predict(X_final_test)
test_scores = final_svm.decision_function(X_final_test)
```

These calls produce two outputs for each of the 319 reviews:

Table 41. Outputs from the fitted model

Output	Meaning	Use in evaluation
Predicted label	0 for truthful or 1 for deceptive	Accuracy, deceptive precision, recall, F1 and macro F1
Decision score	Position relative to the SVM's decision boundary	ROC-AUC, average precision and ranking curves

The notebook verifies that the classifier's class order is $[0, 1]$, so positive decision scores correspond to the deceptive class. The default boundary remains at score zero. These scores are not calibrated probabilities.

Test labels are supplied to the evaluation function to compare predictions with the recorded answers. They are not passed into the classifier's prediction calls, and evaluation does not update the fitted model. Predictions and decision scores are retained for the results and error analysis in Sections 4.3 and 4.4.

Separating diagnostics from final assessment

The same fitted model is also evaluated on its training data. This provides a diagnostic comparison with the held-out results and helps reveal a generalisation gap. Training performance alone cannot establish how well the model predicts reviews from unfamiliar hotels.

Each final test metric is calculated across all 319 reserved reviews. These are single test-set estimates, so the report does not attach the five-fold standard deviations used during cross-validation. The test curves describe the fixed model's behaviour; they are not used to select another threshold.

For inference on a new review, the required processing would remain the same: apply the established cleaning, transform the text with the training-fitted vectoriser, and pass the resulting vector to the fitted SVM. Notebook 3 demonstrates evaluation through saved feature matrices; it does not save a combined raw-text prediction package.

4.3 Final test results

After choosing the final pipeline, I wanted to answer a fairly simple question: could the model recognise deceptive reviews from hotels it had never seen during training?

The final model was a Linear SVM with $C=6$, using non-lemmatised TF-IDF features containing unigrams and bigrams. It was trained on all 1,277 training reviews and evaluated on the 319 reviews reserved for the final test. These reviews came from four previously unseen hotels and included 159 truthful reviews and 160 deceptive reviews.

The model correctly classified 281 of the 319 reviews, giving an accuracy of 88.1%. The following results use the model’s default decision boundary. Neither the model settings nor the decision boundary were adjusted using the test results.

Overall performance

Table 42. Overall training and held-out test performance

Metric	Training set	Held-out test set
Accuracy	1.000	0.881
Deceptive-review precision	1.000	0.918
Deceptive-review recall	1.000	0.838
Deceptive-review F1	1.000	0.876
Macro-F1	1.000	0.881
ROC-AUC	1.000	0.949
Average precision	1.000	0.954

There are two useful things to take from this table. First, the model performed substantially better than a simple constant prediction. Because the test set was almost evenly balanced, predicting every review as deceptive would produce an accuracy of approximately 50.2% and a deceptive-review F1 of 0.668. These are analytical reference values calculated from the class counts, rather than results from an additional trained model.

Second, the perfect training results did not carry over to the unseen hotels. This gap shows why evaluating the model on its training reviews would give an overly optimistic impression of its performance. It does not, by itself, establish the cause of the gap.

The final deceptive-review F1 of 0.876 was also close to the selected configuration’s mean hotel-grouped cross-validation F1 of 0.886 ± 0.030 . That is encouraging, although the two results come from different hotel groups and training sizes. The cross-validation result is an average across five folds; the final test result is one estimate from the reserved test set.

4.3 Final test results (continued)

What the model got right - and what it missed

The class-level results give a clearer picture of how the model behaved.

Table 43. Class-level performance on the held-out test set

Actual review class	Precision	Recall	F1	Number of reviews
Truthful	0.850	0.925	0.886	159
Deceptive	0.918	0.838	0.876	160

The corresponding confusion matrix shows the individual counts behind these scores.

Table 44. Final test confusion matrix

Actual class	Predicted truthful	Predicted deceptive	Total
Truthful	147	12	159
Deceptive	26	134	160
Total	173	146	319

Of the 146 reviews flagged as deceptive, 134 were labelled deceptive in the dataset. This gives a deceptive-review precision of 91.8%. For a hotel using these predictions to decide which reviews deserve closer attention, that precision matters: unnecessary flags can create extra work and cast doubt on genuine customers.

However, the model detected only 134 of the 160 deceptive reviews, giving a recall of 83.8%. It missed 26 deceptive reviews, or 16.3% of that class. It also incorrectly flagged 12 truthful reviews, equivalent to 7.5% of the truthful class.

At the default decision boundary, the model therefore missed more deceptive reviews than it falsely flagged truthful ones. F1 summarises the balance between precision and recall, but the separate error counts remain important because those two mistakes have different practical consequences.

Confusion matrix and score-based evaluation

Figure 10 is taken directly from Notebook 3. Its three panels are displayed separately at a larger size for readability. They bring together the confusion matrix, ROC curve and precision-recall curve.

4.3 Final test results (continued)

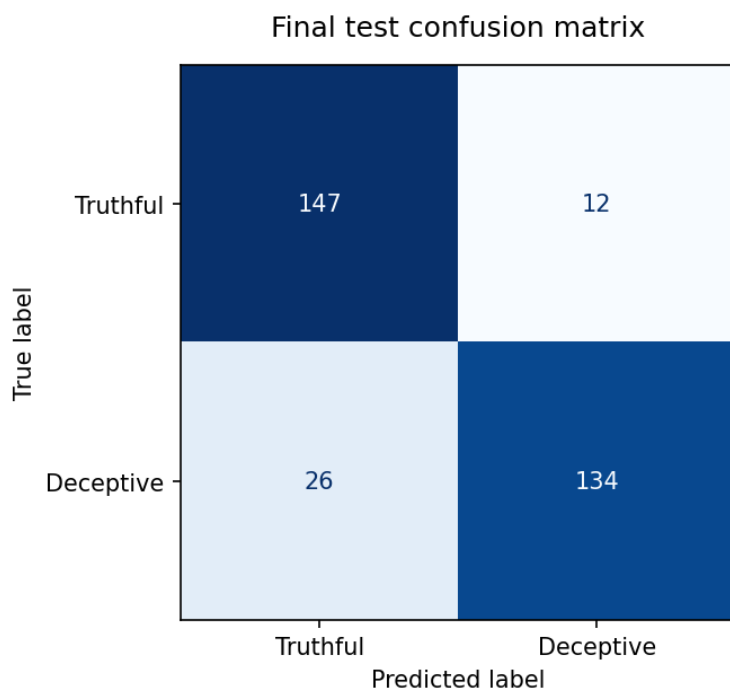


Figure 10a. Final test confusion matrix for the 319 held-out reviews. Original panel from Notebook 3.

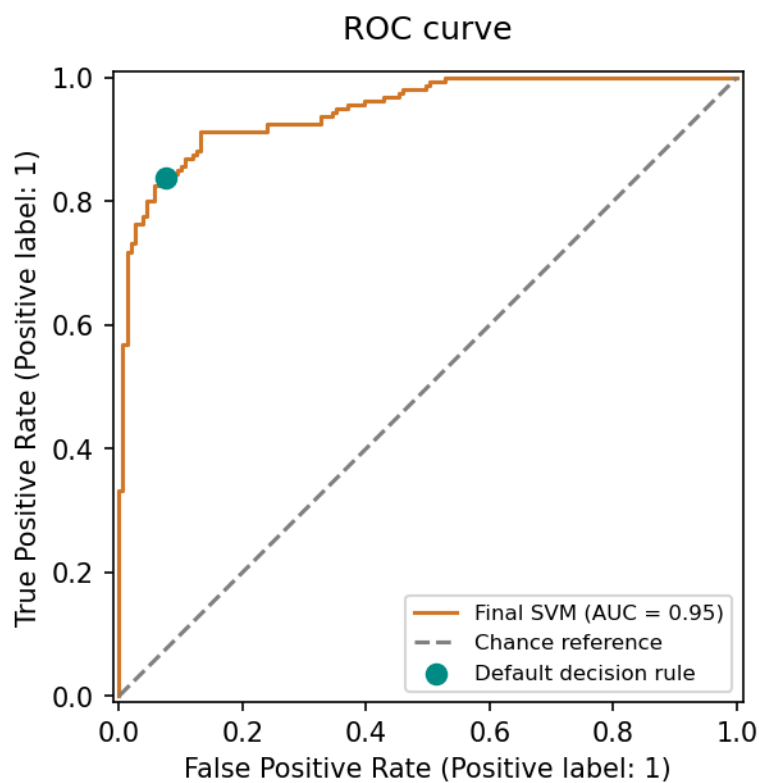


Figure 10b. ROC curve. The marked point shows the default decision threshold of zero. Original panel from Notebook 3.

4.3 Final test results (continued)

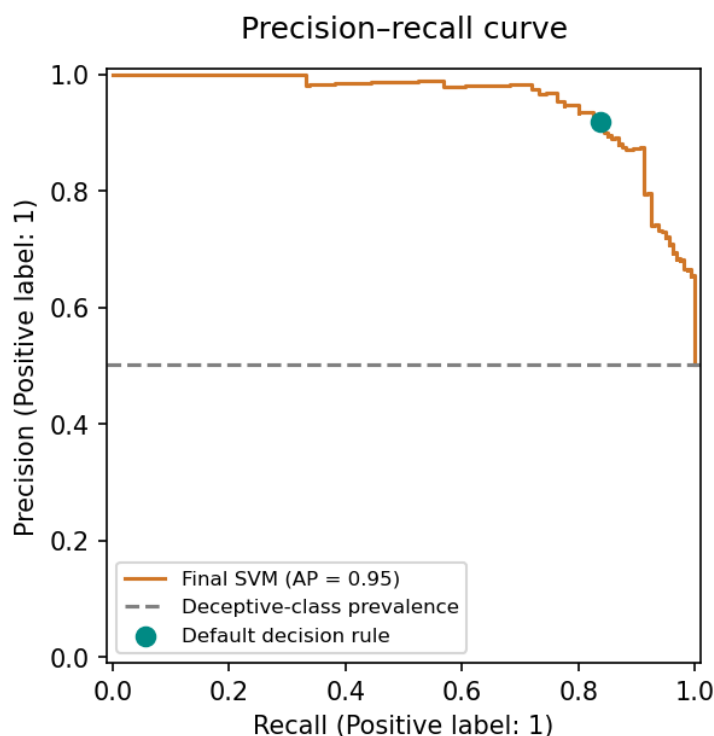


Figure 10c. Precision-recall curve. The marked point shows the default decision threshold of zero. Original panel from Notebook 3.

The ROC-AUC of 0.949 indicates that the model's decision scores generally ranked deceptive reviews above truthful reviews. ROC-AUC can be calculated from the SVM's decision scores; those scores do not need to be probabilities. It describes discrimination across decision thresholds, rather than the accuracy at the selected threshold. [Scikit-learn ROC-AUC documentation](#).

The average precision of 0.954 summarises the precision-recall relationship across score thresholds, weighting precision by increases in recall. The horizontal reference line in the precision-recall plot represents the deceptive-class prevalence, approximately 0.502. [Scikit-learn average-precision documentation](#).

These curves help explain how informative the scores are, but they do not remove the 38 classification errors at the default boundary. They were used to describe the final model's behaviour, without selecting a new threshold from the test set.

4.3 Final test results (continued)

Performance across the held-out hotels

The pooled scores also hide some variation between hotels.

Table 45. Performance across the four held-out hotels

Held-out hotel	Reviews	Truthful falsely flagged	Deceptive reviews missed	Deceptive F1	Macro-F1
Allegro	80	5	4	0.889	0.887
Homewood	80	2	9	0.849	0.861
Knickerbocker	80	2	5	0.909	0.912
Monaco	79	3	8	0.853	0.860

Each hotel contributed 40 deceptive reviews. Allegro, Homewood and Knickerbocker also contributed 40 truthful reviews each, while Monaco contributed 39.

The deceptive-review F1 ranged from 0.849 at Homewood to 0.909 at Knickerbocker. Homewood and Monaco accounted for 17 of the 26 missed deceptive reviews. This suggests that performance was uneven across the four hotels, although these small groups do not establish why the differences occurred.

Overall, the results support the model’s ability to predict the dataset’s review labels for unseen hotels within this corpus. They also support a cautious practical role: using predictions to prioritise reviews for further inspection. A prediction alone cannot establish whether a particular customer is being dishonest, especially given the dataset’s collection-source differences and the limited number of test hotels.

4.4 Error analysis and interpretation

Understanding the two types of error

Beyond the overall results, I examined where the final model’s predictions disagreed with the dataset’s recorded labels. This distinction matters because the two types of mistake have different consequences for ReviewBuster.

A false positive occurs when a review labelled truthful is flagged as deceptive. A false negative occurs when a review labelled deceptive is classified as truthful.

Table 46. Classification errors at the fixed decision threshold

Error type	Reviews	Relevant class total	Error rate
Truthful reviews falsely flagged	12	159 truthful reviews	7.55%
Deceptive reviews missed	26	160 deceptive reviews	16.25%

The model missed more deceptive reviews than it falsely flagged truthful ones. Because the test classes were almost equally sized, this difference also appears in their respective error rates.

These rates use different denominators from precision. For example, the 12 false positives represent 7.55% of truthful reviews, but 8.22% of the 146 reviews flagged as deceptive.

For the project’s intended use, a false positive could lead to an unnecessary investigation or unfair suspicion towards a genuine reviewer. A false negative could allow a misleading review to pass undetected. The model’s output should therefore support further investigation rather than establish whether a reviewer was dishonest.

Where errors occurred across hotels

I then examined the saved predictions separately for each held-out hotel. This helped identify whether the overall result concealed groups with more frequent errors.

Table 47. Error counts and deceptive F1 across the four test hotels

Hotel	Truthful reviews	Deceptive reviews	False positives	False negatives	Deceptive F1
Allegro	40	40	5	4	0.889
Homewood	40	40	2	9	0.849
Knickerbocker	40	40	2	5	0.909
Monaco	39	40	3	8	0.853

4.4 Error analysis and interpretation (continued)

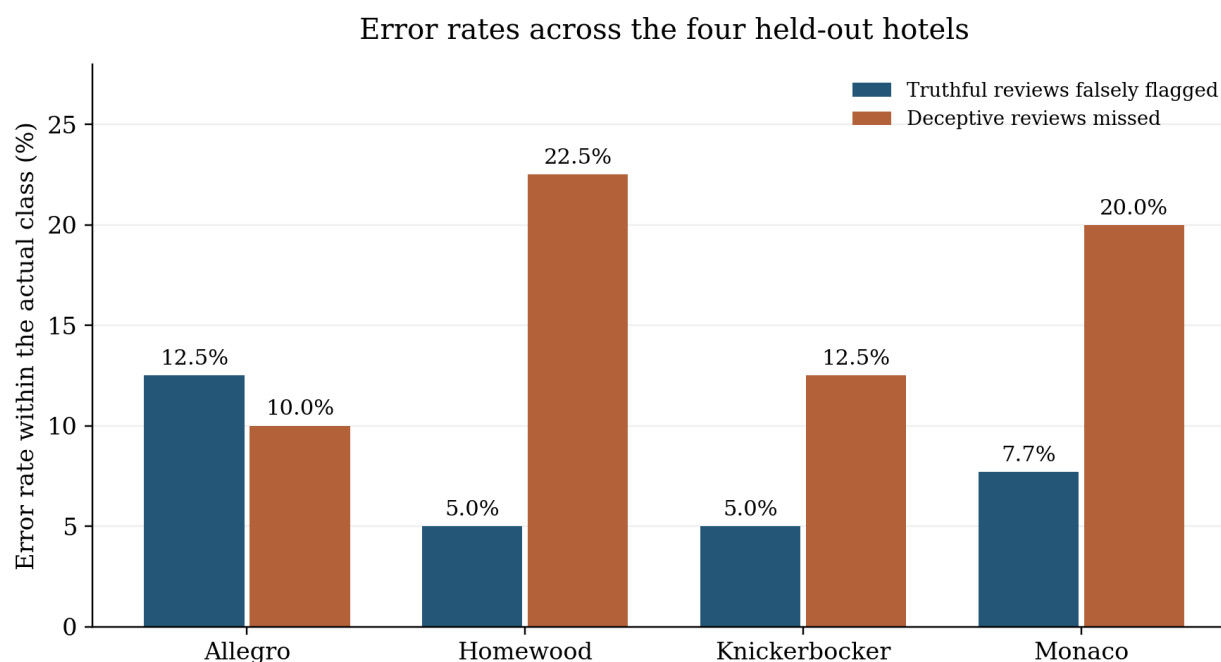


Figure 11. Hotel-level error rates calculated from Notebook 3’s saved counts. False-flag rates use each hotel’s truthful-review total; missed-deception rates use its deceptive-review total.

Homewood and Monaco contributed 17 of the 26 false negatives, despite containing only half of the deceptive test reviews. Their missed-deception rates were 22.5% and 20.0%, respectively.

Allegro showed a different pattern. It had the largest number of false positives, incorrectly flagging five of its 40 truthful reviews, but missed only four deceptive reviews. This illustrates why a single overall score cannot fully describe the model’s behaviour.

These results identify where errors were concentrated in this test split. They do not establish why particular hotels were harder. Each hotel contributed only 79 or 80 reviews, so the comparisons provide descriptive evidence rather than a reliable ranking of future performance.

Inspecting individual mistakes

Notebook 3 displays three examples of each error type. The examples were selected by their strongest decision scores in the wrong direction: the highest positive scores among false positives and the most negative scores among false negatives.

This selection highlights substantial disagreements with the recorded labels. It is not a random or representative sample of all errors.

The notebook also checks the alignment between the original review text and the saved test metadata before attaching readable text. This ensures that each displayed review corresponds to the prediction being inspected.

4.4 Error analysis and interpretation (continued)

Truthful reviews flagged as deceptive

Table 48. Selected false positives from the saved notebook output

Hotel	Decision score	Review content
Monaco	+0.684546	A family visit, plans to return, and children enjoying the room's goldfish and window seats.
Homewood	+0.400182	A visit with teenage daughters, breakfast and receptions, nearby activities, and using the hotel dryer after a thunderstorm.
Monaco	+0.209103	Praise for the location, service, room design, wireless internet and goldfish.

Review descriptions are paraphrased.

All three examples are favourable accounts containing personal experiences or specific hotel details. Nevertheless, the model placed them on the deceptive side of its decision boundary.

The two Monaco examples both mention goldfish. This is an observable similarity between the selected errors, but it does not establish that this term caused either prediction. Determining which features contributed to the decisions would require inspecting the fitted coefficients and each review's feature values.

These examples show that personal anecdotes and distinctive details can occur in truthful reviews that the model misclassifies. They do not establish that favourable sentiment generally causes false positives.

Deceptive reviews classified as truthful

Table 49. Selected false negatives from the saved notebook output

Hotel	Decision score	Review content
Allegro	-0.679163	Complaints about poor lighting and an unresolved request to reception.
Homewood	-0.641688	A frequent business traveller praising the rooms, beds, prices and convenient location.
Knickerbocker	-0.637902	Detailed complaints about room size, bathrobes, slippers and treatment by reception staff.

Review descriptions are paraphrased.

These reviews also contain plausible experiences and concrete details, despite their deceptive labels. The selected examples include both praise and complaints, illustrating that deceptive content in this corpus is not confined to one sentiment.

Together, the false positives and false negatives demonstrate overlap in the language of the two classes. Detailed descriptions alone do not reliably distinguish them.

The TF-IDF representation captures words and short phrases, but these examples do not reveal whether a specific phrase, a combination of features or source-related writing patterns produced each mistake. Any explanation of the precise cause would require further analysis.

4.4 Error analysis and interpretation (continued)

Interpreting the decision scores

The displayed errors are not all close to the decision boundary. For example, the strongest selected false positive has a score of +0.684546, while the strongest selected false negative has a score of -0.679163.

These values show the direction and strength of the model’s decision relative to its boundary. They are not calibrated probabilities and should not be presented as percentages of confidence that a review is deceptive.

The examples also indicate that changing the threshold would involve a trade-off. Lowering it could detect additional deceptive reviews while increasing false flags. Raising it could reduce false flags while missing more deceptive reviews. The threshold remained fixed for the reported test assessment.

What the errors mean for ReviewBuster

The SVM’s squared-hinge training objective penalises margin violations. It does not directly encode the practical costs of wrongly accusing a genuine reviewer or allowing a deceptive review to pass.

The selected errors make that gap concrete: truthful accounts were flagged, while plausible accounts labelled deceptive were accepted. Error counts, class-specific rates and readable examples therefore provide information that the training objective or a single F1 score cannot supply.

Finally, these disagreements are measured against the corpus’s recorded labels. Source-related writing patterns may remain in the text even though source metadata was excluded from prediction. The analysis supports conclusions about the model’s behaviour on these four test hotels, while leaving the underlying causes of individual mistakes unresolved.

4.5 Discussion: practical objective, limitations, and future work

From label prediction to the practical objective

ReviewBuster began with a practical question: could review text help identify potentially deceptive hotel reviews? The experiment approaches this question by predicting the truthful and deceptive labels supplied by the dataset.

In a practical setting, the desired outcome would be to help a reader or moderator identify misleading reviews while avoiding unnecessary suspicion towards genuine reviewers. Correctly predicting a dataset label provides evidence relevant to that goal, but it does not independently establish whether a review describes a real stay or whether its author was dishonest.

During training, the recorded labels guide the model’s loss calculation. During validation and testing, predictions are compared with those labels. The project therefore measures agreement with the corpus’s classification of reviews, rather than independently verifying the events described in them.

For this reason, I would position ReviewBuster as a tool for prioritising reviews for further investigation. Assessing its practical usefulness would also require evidence about the quality of those investigations, the number of unnecessary flags and the workload created for moderators. These outcomes were not measured in the current experiment.

Why the training objective is an approximation

The SVM minimises a regularised squared-hinge objective, while model selection uses deceptive F1. Neither directly represents the practical consequences of each decision.

Squared hinge loss penalises incorrect predictions and insufficient classification margins. It provides a tractable objective for learning a separating boundary. Deceptive F1 then evaluates the balance between precision and recall using completed class predictions.

I accepted this difference because the training objective supports fitting the classifier, while validation metrics and the error analysis provide additional checks on its behaviour. However, those checks do not remove the mismatch. A setting that improves F1 could still produce a less desirable balance of errors for a particular moderation workflow.

Section 4.4 provides concrete examples: some truthful accounts were flagged, while some plausible accounts labelled deceptive were accepted. These mistakes have different consequences, and the current objective does not assign explicit costs to them.

A future study could define a validation criterion such as:

$$E_{\text{cost}} = \frac{c_{\text{FP}} \text{FP} + c_{\text{FN}} \text{FN}}{N}$$

Here, c_{FP} represents the cost of a false flag, c_{FN} represents the cost of missed deception, and N is the number of evaluated reviews.

This would be a proposed decision criterion, not the loss used in the current implementation. Its costs would need to be justified for the intended use. They should not be assigned arbitrary values simply to favour a particular result. Threshold selection can then be evaluated against that criterion using validation data. See the `scikit-learn` cost-sensitive threshold example.

What the results establish

The reserved hotel test provides evidence that the selected model learned patterns that transferred beyond its training hotels. Nevertheless, successful classification on this split does not establish general performance across the hotel industry.

4.5 Discussion: practical objective, limitations, and future work (continued)

The final model classified its training data perfectly, while test accuracy was approximately 0.881. This demonstrates a generalisation gap and reinforces why training performance alone was insufficient for evaluating the project.

The test deceptive F1 was also approximately 0.010 below the selected cross-validation mean. Those scores involve different review groups, training sizes and aggregation methods. Their difference alone does not establish a statistically reliable decline.

The ranking results similarly need a practical interpretation. Strong separation of decision scores does not determine which threshold best suits a moderation process, nor does it turn those scores into probabilities of deception. The project retained its original decision rule for the final assessment.

Limitations of the evidence

Review source and class label are confounded. As identified during data exploration, MTurk reviews are labelled deceptive, while TripAdvisor and Web reviews are labelled truthful. Although source metadata was excluded from prediction, source-related writing patterns can remain in the text. Consequently, some predictive performance may reflect differences in how the reviews were collected. Hotel grouping does not resolve this limitation.

The evaluation covers a limited setting. Final testing involved four hotels and 319 reviews. The differences between hotels reported in Section 4.4 show variation within that split, but do not establish which hotels or review styles will consistently be harder. Applying the model to the Vietnamese hospitality context that motivated the project would require separate evaluation using appropriate data.

Validation also influenced selection. The same five hotel-grouped folds supported model comparison and subsequent tuning. Selecting the strongest result across many candidates can make the selected validation score optimistic. The reserved test set provides a separate assessment of the fixed configuration, while nested cross-validation would offer a stronger assessment of the complete selection procedure. See the [scikit-learn nested cross-validation example](#).

The reasons for individual predictions remain partly unresolved. The selected mistakes demonstrate overlapping language between the classes, but the notebook does not provide feature-contribution analysis. The six displayed errors were also chosen by their strongest scores in the wrong direction. They cannot establish the frequency of particular linguistic patterns across all errors.

Practical operation has not been evaluated. Notebook 3 demonstrates training and testing through saved feature matrices. It does not provide a deployed service or a saved package combining raw-text preprocessing, the fitted vectoriser and the classifier. Prediction latency, operating workload and performance on subsequently collected reviews were not benchmarked.

The experiment also concerns this human-written review corpus. It does not establish performance on AI-generated reviews.

4.5 Discussion: practical objective, limitations, and future work (continued)

Future work guided by these limitations

The next steps should address the weaknesses identified by the evaluation, rather than assume that a more complicated model will necessarily perform better.

Table 50. Proposed improvements and the evidence needed to evaluate them

Limitation or finding	Proposed next step	How it should be assessed
Source is closely associated with the label	Collect data containing both classes within comparable collection sources, with documented label evidence	Evaluate on independent hotels and sources to examine transfer beyond collection patterns
Missed deception remains frequent	Compare class weighting and alternative decision thresholds	Use training-only hotel-grouped validation, monitoring both missed reviews and false flags against a stated practical criterion
Performance varies between hotels	Expand the number and variety of hotels and use nested hotel-grouped validation	Report group-level results and assess the complete selection process on outer held-out groups
Causes of individual mistakes are unresolved	Inspect feature contributions and sample errors more systematically	Compare explanations across false positives, false negatives and correctly classified reviews
Decision scores are not probabilities	Assess calibration if a future interface requires probability estimates	Use held-out or cross-validated training predictions, preserving hotel separation
The implementation stops at notebook evaluation	Package preprocessing, the fitted vectoriser and classifier into one prediction workflow	Check consistent raw-text processing, feature alignment and operating performance on new data

Calibration would require data separated appropriately from classifier fitting; it should not be fitted using the final test labels. See the [scikit-learn calibration documentation](#).

Any threshold changes would likewise need validation data separate from the observations used to fit the corresponding classifier. See the [scikit-learn threshold-tuning documentation](#).

These proposals have not been implemented or evaluated in the current project. Because the existing test results have now been inspected, a future version developed in response to these findings should receive a new independent final assessment.

4.6 Conclusion

ReviewBuster began with a question that came from my family’s experience in hospitality: could the language in a review help identify something that deserves a closer look? This project narrowed that question into an experiment - whether a machine learning model could distinguish truthful from deceptive reviews when those reviews came from hotels it had not seen during training.

I compared logistic regression, a linear SVM and a small neural network using hotel-grouped cross-validation. The comparisons supported non-lemmatised TF-IDF unigrams and bigrams with a linear SVM as the primary pipeline. Adding neighbouring word pairs provided useful information in the tested configurations, while lemmatisation offered no consistent improvement. Further validation selected $C=6$ as the final regularisation setting. These choices were made using the training reviews, before examining the final test results.

After training on all 1,277 training reviews, the final model correctly classified 281 of 319 reviews from four previously unseen hotels. It achieved 88.1% accuracy, 0.876 deceptive-review F1 and 0.881 macro-F1. Its deceptive-review precision was 91.8%, while recall was 83.8%. These results provide evidence that the selected pipeline could predict the dataset’s labels for unfamiliar hotels within this corpus.

The mistakes also help explain what those results mean. The model detected 134 of 160 deceptive reviews, but missed 26 and incorrectly flagged 12 truthful reviews. At its fixed decision boundary, missed deception was therefore the more frequent error. Perfect training performance did not eliminate these mistakes on unseen hotels, and the inspected examples showed that plausible details and personal experiences could appear in both classes.

For a hotel, those errors have consequences that a single score cannot fully describe. A false flag could create unnecessary work or unfair suspicion towards a genuine customer; a missed deceptive review could leave misleading information unchallenged. The SVM’s regularised squared-hinge objective does not directly encode these practical costs. Precision, recall and error inspection therefore remain necessary checks on whether the model is useful for the intended task.

There is also a limit to the answer this project can offer. The evaluation covers one human-written review corpus and four held-out hotels. Collection sources are closely associated with the labels, so the model may partly recognise source-related writing patterns. The results do not establish performance on Vietnamese hotels, AI-generated reviews or every future review platform.

A useful next step would be to assess the pipeline on independently collected reviews with less dependence between source and label. Class weighting or a different decision threshold could also be investigated if missed deception is particularly costly, using training-only hotel-grouped validation followed by a new independent test.

For now, ReviewBuster provides evidence that a relatively simple text classifier can help prioritise reviews for inspection. That brings the project back to its original motivation: giving people working in hospitality another piece of information when deciding which reviews need closer attention.

5 Implementation Log

5.1 Development decisions, challenges, and solutions

The biggest difficulty for me was understanding how to apply the machine learning framework to the problem I wanted to solve. I had a practical goal for ReviewBuster, but I needed to understand how the model’s inputs, training objective and evaluation would connect to that goal.

I worked through this by researching the concepts and revisiting the lecture slides week by week. This helped me connect the framework to the implementation: review text becomes numerical features, the model learns from labelled examples, and its predictions are compared with the recorded labels. I also needed to distinguish the objective used during training from the measures used to assess whether the model was useful.

That distinction became particularly relevant when evaluating the final SVM. Its regularised squared-hinge objective guides the learning process, while the practical concern is identifying potentially deceptive reviews without creating too many false flags. I therefore used deceptive-review F1 for model selection and examined precision, recall and the confusion matrix to understand the different mistakes.

The project’s validation design also supported this goal. Hotel-grouped cross-validation assessed reviews from hotels excluded from each training run, while four separate hotels were reserved for final testing. This connected the evaluation to the research question about unfamiliar hotels.

I made the important project decisions myself. Codex acted as a discussion partner, helping me examine the reasoning behind choices such as text representation, model comparison and hyperparameter refinement. The final decisions were informed by those discussions, the lecture material and the results recorded in the notebooks.

5.2 Use of AI tools and verification

I used Codex throughout the project to help study concepts, generate code and refine parts of the written explanations. I treated it as a discussion companion: when a topic was difficult, I could ask for a more detailed explanation and continue the discussion until the reasoning became clearer.

My review involved going through the generated work again and studying the code to understand what it was doing. This included following how the data entered the model, how training and prediction were separated, and how the evaluation outputs related to the project’s goal. Understanding these connections was an important part of checking the assistance I received.

One documented example of correcting AI output occurred during report preparation. Codex initially included a table analysing the relationship between collection source and review label in Section 2.1. I asked for that discussion to be moved to Section 2.2, keeping Section 2.1 focused on the dataset background and attributes. This preserved a clearer progression from describing the dataset to interpreting its patterns.

AI assistance contributed to the implementation and writing, while I retained responsibility for the project’s design choices and for understanding the submitted work.

5 Implementation Log (continued)

5.3 Knowledge gaps and attempts to resolve them

Some of the evaluation metrics were difficult for me to grasp fully. I needed to understand what each measure revealed about the model's behaviour and why several measures were useful for the same task.

I used AI as a learning companion and asked for detailed explanations. The project's final results provide a concrete way to distinguish the measures. Precision considers the reviews flagged as deceptive: 134 of the 146 flags matched the dataset's deceptive label. Recall considers all deceptive reviews: the model detected 134 of 160. These measures therefore describe different questions, even though they use the same predictions.

F1 brings precision and recall together, while the confusion matrix exposes the individual error counts. ROC-AUC and average precision examine the model's decision scores across thresholds, rather than only its predictions at the fixed boundary.

This area mattered because interpreting the results required more than recognising whether a score was high. For ReviewBuster, the measures needed to explain both how often a flag was correct and how much deceptive content the model missed. Asking for detailed explanations helped me connect the evaluation terminology to those practical concerns.

References

Sources cited or named in the report. Repeated citations are listed once. Web sources were accessed on 1 October 2026; n.d. indicates that no publication date is stated.

LIBLINEAR project (n.d.). **LIBLINEAR README**. Software documentation. GitHub, cjin1/liblinear.
<https://github.com/cjin1/liblinear/blob/master/README>

NLTK developers (n.d.). **nlk.stem.wordnet module: WordNetLemmatizer**. NLTK API documentation.
<https://www.nltk.org/api/nltk.stem.wordnet.html>

NLTK developers (n.d.). **nlk.tag package: part-of-speech tagging**. NLTK API documentation.
<https://www.nltk.org/api/nltk.tag.html>

nvmngo (n.d.). **ReviewBuster_NLP_MLproject**. Project source code and Jupyter notebooks. GitHub.
https://github.com/nvmngo/ReviewBuster_NLP_MLproject

Ott, M., Choi, Y., Cardie, C. and Hancock, J.T. (2011). **Finding Deceptive Opinion Spam by Any Stretch of the Imagination**. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pp. 309-319. Association for Computational Linguistics.
<https://aclanthology.org/P11-1032/>

Ott, M., Cardie, C. and Hancock, J.T. (2013). **Negative Deceptive Opinion Spam**. Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 497-501. Association for Computational Linguistics.
<https://aclanthology.org/N13-1053/>

scikit-learn developers (n.d.). **__multilayer_perceptron.py**. Neural network implementation, source code at revision 98ed9dc73. GitHub, scikit-learn/scikit-learn.
https://github.com/scikit-learn/scikit-learn/blob/98ed9dc73/sklearn/neural_network/__multilayer_perceptron.py

scikit-learn developers (n.d.). **average_precision_score**. scikit-learn 1.6.1 documentation.
https://scikit-learn.org/1.6/modules/generated/sklearn.metrics.average_precision_score.html

scikit-learn developers (n.d.). **CalibratedClassifierCV**. scikit-learn 1.6.1 documentation.
<https://scikit-learn.org/1.6/modules/generated/sklearn.calibration.CalibratedClassifierCV.html>

scikit-learn developers (n.d.). **GroupKFold**. scikit-learn 1.6.1 documentation.
https://scikit-learn.org/1.6/modules/generated/sklearn.model_selection.GroupKFold.html

scikit-learn developers (n.d.). **Linear models: logistic regression**. scikit-learn 1.6.1 documentation.
https://scikit-learn.org/1.6/modules/linear_model.html#logistic-regression

References (continued)

scikit-learn developers (n.d.). **LinearSVC**. scikit-learn 1.6.1 documentation.

<https://scikit-learn.org/1.6/modules/generated/sklearn.svm.LinearSVC.html>

scikit-learn developers (n.d.). **LogisticRegression**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/modules/generated/sklearn.linear_model.LogisticRegression.html

scikit-learn developers (n.d.). **MLPClassifier**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/modules/generated/sklearn.neural_network.MLPClassifier.html

scikit-learn developers (n.d.). **Nested versus non-nested cross-validation**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/auto_examples/model_selection/plot_nested_cross_validation_iris.html

scikit-learn developers (n.d.). **Neural network models (supervised)**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/modules/neural_networks_supervised.html

scikit-learn developers (n.d.). **Pipeline**. scikit-learn 1.6.1 documentation.

<https://scikit-learn.org/1.6/modules/generated/sklearn.pipeline.Pipeline.html>

scikit-learn developers (n.d.). **Post-tuning the decision threshold for cost-sensitive learning**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/auto_examples/model_selection/plot_cost_sensitive_learning.html

scikit-learn developers (n.d.). **roc_auc_score**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/modules/generated/sklearn.metrics.roc_auc_score.html

scikit-learn developers (n.d.). **Support vector machines**. scikit-learn 1.6.1 documentation.

<https://scikit-learn.org/1.6/modules/svm.html>

scikit-learn developers (n.d.). **TfidfVectorizer**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

scikit-learn developers (n.d.). **TunedThresholdClassifierCV**. scikit-learn 1.6.1 documentation.

https://scikit-learn.org/1.6/modules/generated/sklearn.model_selection.TunedThresholdClassifierCV.html

Tatman, R. (2017). **Deceptive Opinion Spam Corpus**. Kaggle dataset distribution of the research corpus.

<https://www.kaggle.com/datasets/rtatman/deceptive-opinion-spam-corpus>